LUISS

Degree Program in
Management and Computer Science

Course of Data Analysis for Business

Neural SDEs for Financial Market Modeling: Implementation and Performance Analysis

Prof. Alessia Caponera

SUPERVISOR

Paola Pinna ID 284961

CANDIDATE

Academic Year 2024/2025

Acknowledgements

I would like to sincerely thank Professor Alessia Caponera for her valuable support and for trusting me with the opportunity to develop this work under her supervision.
I would also like to mention the meaningful collaboration with Professor Francesco Iafrate who, despite not being my official co-supervisor, has closely followed and supported me throughout the writing of this thesis. I am sincerely grateful to him for his guidance, availability, and thoughtful advice.

Table of Contents

Introduction: Context and Objectives *4*

Chapter 1: Theoretical Foundations of Neural SDEs *7*

1.1 Neural SDEs as an Extension of Neural ODEs **8**

1.2 Mathematical Preliminaries for Stochastic Differential Equations **9**

Brownian Motion as a Building Block of SDEs 9
Itô Calculus: The Language of Stochastic Dynamics 11
What is Ito lemma? 12
Stratonovich Integral: An Alternative Formulation of the Stochastic Integral 13

1.3 Neural Stochastic Differential Equations: Formulation and Core Properties **14**

Existence and Uniqueness of Solutions 15
Universal Approximation Capabilities 16
Numerical Methods for Simulating Neural SDEs 17

Chapter 2: Literature Review *19*

2.1 Evolution of Stochastic Modeling in Finance: From Classical to Neural Approaches **19**

2.2 Generative Adversarial Networks (GANs) for Sequential Data **22**

2.3 Neural SDEs: Training Paradigms and Integration into GANs **23**

Chapter 3: Methodology *27*

3.1 Data Acquisition and Preprocessing **27**

Data Source and Description 27
Initial Data Transformation 28
Distribution Analysis 29
Outlier Handling 29
Data Scaling 30
Sequence Generation 30

3.2 SDE-GAN Model Architecture **30**

Generator 32
Discriminator (Critic) 35

3.3 Training Procedure **37**

Gradient Penalty 37
Training Loop 37
Training Dynamics and Monitoring 39

3.4 Hyperparameters Tuning **39**

Chapter 4: Empirical Evaluation *42*

4.1 Evaluation Metrics **42**

4.2 Evaluation Setup and Sample Extraction **43**

4.3 Evaluation on Train Data **44**

4.4 Evaluation on Test Data **48**

Chapter 5: Discussion and Future Research Directions *52*

5.1 Navigating Computational Complexity and Training Instability **52**

5.2 Future Improvements and Directions **53**

Introduction

Context and Objectives

The ability to generate realistic, data-driven synthetic versions of market behavior isn't just an interesting academic problem; it is a practical and powerful resource for financial institutions to manage uncertainty. By simulating plausible future asset dynamics, generative models can offer valuable support for critical decision-making across a wide range of financial applications.

Take risk management for example. With better synthetic scenarios, firms can test more thoroughly how their investment portfolios behave. This means exposing them to a wider range of possible market movements than historical data alone can offer, even extreme ones never seen before. That’s what allows to, for instance, make a more robust assessment of risk measures such as Value-at-Risk (VaR) and Expected Shortfall (ES), particularly in the context of complex portfolios with non-linear and path-dependent exposures to market factors. At the same time, it helps ensure that banks and investment firms are better prepared for severe conditions.

Algorithmic trading is one clear example of how crucial risk assessment has become in financial practice. Trading algorithms must be rigorously stress-tested across a wide range of potential market paths to ensure robustness and stability beyond historical performance. Synthetic data makes it easier to judge how consistently a strategy performs over time.

Moreover, realistic synthetic paths are extremely useful for the rigorous testing and refinement of hedging strategies. Options portfolios, for example, are typically managed using dynamic hedging rules that trigger rebalancing based on market movements, which are influenced by realized volatility and the underlying asset's actual price trajectory (including its effective drift). By simulating a wide array of plausible paths, including scenarios with different volatility regimes, plausible real-world drift assumptions, and sudden price jumps, practitioners can more thoroughly assess hedge performance, identify vulnerabilities, quantify residual risk, and evaluate the impact of transaction costs, thereby leading to robust risk management and improved cost-efficiency.

In the world of derivative pricing, particularly for exotic instruments whose values depend on the entire path of an underlying asset, closed-form solutions are often elusive. Simulating asset paths under the requisite risk-neutral measure ($\mathbb{Q}$), becomes pivotal. The accuracy of valuations and hedges then critically depends on the ability of the underlying models to generate such risk-neutral paths reflecting market dynamics.

Despite these needs, traditional econometric and stochastic models, while foundational, often make simplifying assumptions, in a way that don't quite capture the full, non-linear dance of real-

world asset prices. The rise of machine learning, especially deep learning, has certainly opened new doors for understanding financial data, allowing models to learn intricate patterns and non-linear relationships directly from vast datasets without the pre-specified assumptions common in traditional approaches.
Finance has been a significant area of interest for machine learning for over four decades, but despite this long history, the field remains full of open questions and exciting opportunities for further research.

Many current ML methods are formulated in discrete time, learning point to point transitions. While practical for many forecasting tasks, they may face limitations when modeling financial processes that are better described by continuous-time stochastic dynamics. This highlights the need for hybrid models that combine machine learning's ability to uncover complex data patterns, with the foundational structure of stochastic calculus, a framework that offers profound insights and powerful tools for modeling price evolution, even when approximating inherently discrete market movements as continuous.

Neural Stochastic Differential Equations (Neural SDEs) are a prime example of this paradigm. While their implementation requires time discretization via numerical solver, their formulation is grounded in continuous-time stochastic processes, enabling them to describe how uncertainty evolves over time in a principled manner.

This work in particular addresses this challenge of modeling and simulating financial time series by developing a Neural Stochastic Differential Equation based model trained with a Generative Adversarial Network framework (SDE-GANs). The idea isn't to predict the precise future price, instead, the aim is to learn the underlying engine, the stochastic process, that drives how asset prices evolve through time. By using flexible neural networks to define the key parts of a stochastic differential equation, and by training this whole system in an adversarial way, the prospect of this research is to see if it's possible to generate very realistic and statistically consistent trajectories. The approach is applied to S&P 500 log-returns, a major financial benchmark, and evaluated using both distributional and temporal metrics.

This thesis therefore seeks to explore several key lines of inquiry:

- Whether an SDE-GAN setup, incorporating Itô calculus and a CDE-based discriminator, can effectively learn the underlying data-generating process of a major financial index.

- The extent to which synthetic trajectories generated by the model replicate key stylized facts of financial returns, including their characteristic marginal distributions, volatility patterns, and temporal dependencies when analyzed as complete paths.
- How well the model generalizes when presented with new, unseen data, hence determining if it has learned robust patterns rather than merely overfitting to the training dataset.
- What computational and practical challenges arise in implementing and training Neural SDEs using the chosen GAN framework, especially concerning memory efficiency, training time, and the stability of adversarial learning dynamics.

To explore these questions, this thesis is laid out as follows:

Chapter 1 covers the essential mathematics, from basic differential equations to Stochastic Differential Equations (SDEs), the details of Itô calculus, and how neural networks fit into this picture.

Chapter 2 looks at where this research fits in. It traces the path from classic SDE models in finance to newer neural network approaches, including Neural ODEs and SDEs, and also reviews how Generative Adversarial Networks have been used for sequential data.

Chapter 3 describes the methodology, how the SDE-GAN model is built in detail: how data was prepared, the specific design of the Neural SDE generator and the CDE-based discriminator, and how the adversarial training actually works.

Chapter 4 presents the results. It examines how well the model performed on historical S&P 500 data, comparing the generated trajectories to the real ones using various statistical measures on both training and test data.

Chapter 5 steps back to discuss what the findings mean, the limitations of this approach, and where future research could go.

The goal of this research is to explore the potential of this deep learning framework as a first step toward developing useful tools for quantitative finance. By investigating its ability to generate realistic financial time series, the study aims to contribute to better risk assessment practices and inspire new directions in model-based financial analysis.

Chapter 1: Theoretical Foundations of Neural SDEs

The accurate modeling of systems that evolve continuously over time and are subject to inherent stochasticity is a central challenge across diverse disciplines. Within quantitative finance, the primary focus of this work, these challenges are particularly pronounced when modeling asset price dynamics. Understanding the mathematical framework that governs such processes is therefore of primary importance, since it provides the language and tools for proper analysis, prediction, and intervention. This chapter lays the theoretical foundations for Stochastic Differential Equations (SDEs) and their subsequent integration with neural networks to form Neural SDEs.

Continuous dynamic systems are mathematically described by differential equations. When a system's evolution is entirely determined by its current state, without the influence of random factors, *Ordinary Differential Equations* (ODEs) provide the appropriate descriptive framework. An ODE, often written as $\frac{dX_t}{dt} = f(X_t, t)$, defines the precise rate of change of a state X_t, allowing for the computation of a unique trajectory given an initial condition $X_{t_0} = X_0$.
However, a vast array of real-world phenomena, from particle movements in physics to asset price fluctuations in finance and latent variable dynamics in machine learning, are not purely deterministic. They are continuously influenced by random perturbations and unpredictable factors that cannot be ignored for faithful representation. To capture these essential characteristics, *Stochastic Differential Equations (SDEs)* extend the deterministic framework. A classic SDE takes the form:

$$dX_t = f(X_t, t)dt + g(X_t, t)dW_t.$$

Here, $f(X_t, t)$ is the drift coefficient, dictating the deterministic part of the evolution; $g(X_t, t)$ is the diffusion coefficient, scaling the intensity of the noise; and dW_t represents an infinitesimal increment of a Wiener process, which models the random influences.

A primary challenge in applying SDEs to complex, real-world data is that the functional forms of f and g are often unknown. This is where the Neural Networks come into play. By leveraging their universal approximation capabilities, it is possible to parameterize these unknown functions, allowing them to be learned from data.
The progression from discrete-time models to continuous-time deterministic models (Neural ODEs) and further to continuous-time stochastic models (Neural SDEs) reflects a significant trend in machine learning. We started with discrete-time models that hop from one data point to the next, then discovered that Neural ODEs let us fill in all the gaps by learning a smooth

trajectory, and now Neural SDEs go even further by embracing the fact that real-world signals aren't just continuous, they're also noisy.

1.1 Neural SDEs as an Extension of Neural ODEs

Neural Ordinary Differential Equations (NODEs), provide a framework for learning continuous latent dynamics by parameterizing the vector field of an ordinary differential equation using a neural network. The evolution of a latent state X_t is described by:

$$\frac{dX_t}{dt} = f\left(t, X_t;\ \theta_f\right)$$

In this framework, a neural network learns to predict the instantaneous slope and velocity of the hidden state, so that the latent vector X_t evolves according to an ordinary differential equation. Equivalently, the solution can be written in integral form as:

$$X_t = X_0 + \int_0^t f\left(s, X_s;\ \theta_f\right) ds$$

This solution represents a deterministic path through the latent space.

Neural SDEs extend this deterministic framework by incorporating a diffusion term, which introduces stochasticity into the latent dynamics, typically driven by a Wiener process (Brownian motion) W_t. This allows Neural SDEs to model not just the mean evolution but a distribution over possible trajectories, thereby capturing uncertainty.

Incorporating stochasticity into a neural continuous-time model isn't that simple, it introduces substantial additional complexity. Neural ODEs operate within deterministic calculus, leveraging established solvers and the adjoint-ODE method for efficient gradient computation, whereas Neural SDEs demand stochastic-calculus machinery, such as, Itô integrals, Itô's lemma, and specialized numerical schemes such as Euler–Maruyama or Milstein, each with its own strong vs. weak convergence and stability guarantees. Parameterizing the diffusion term with a neural network further necessitates architectural and regularization choices to ensure the SDE remains stable and avoids explosive trajectories. Gradient computation likewise becomes more involved, requiring stochastic adjoint techniques that solve a backward SDE rather than a simple reverse-ODE. As a result, the successful application of Neural SDEs depends on carefully designing drift and diffusion networks, selecting appropriate integrators, and employing robust training strategies to achieve stable and accurate modeling of noisy, irregular data.

1.2 Mathematical Preliminaries for Stochastic Differential Equations

A rigorous understanding of Neural SDEs necessitates a solid grounding in the theory of stochastic processes and stochastic calculus.

Brownian Motion as a Building Block of SDEs

Brownian motion is the very first important concept characterizing Stochastic Differential Equations, also called Wiener process, it underlies the most classical stochastic calculus results, and it is fundamental to model random continuous evolution. It is defined as a continuous-time Gaussian process W_t with the following properties:

Initial State: $W_0 = 0$, the process starts at the origin.

Continuous Paths: With probability 1, the function $t \mapsto W_t$ is continuous in t. This implies that the sample trajectories of a Wiener process do not exhibit sudden jumps or discontinuities (see *Figure 1*).

Independent Increments: For any finite set of time points $0 \leq s_1 < t_1 \leq s_2 < t_2 \leq \cdots \leq s_n < t_n$, the random variables representing the increments over non-overlapping intervals, $W_{t_1} - W_{s_1}$, $W_{t_2} - W_{s_2}, \ldots,\ W_{t_n} - W_{s_n}$, are mutually independent. This property captures the notion that future movements of the process are independent of past movements, given the present state.

Stationary Increments: For any $s, t \geq 0$, the distribution of the increment $W_{t+s} - W_s$ is the same as the distribution of $W_t - W_0 = W_t$. This means the statistical properties of the change in the process depend only on the duration of the time interval, not on its starting point.

Normal Distribution of Increments: For any $0 \leq s < t$, the increment $W_t - W_s$ is normally distributed with mean 0 and variance $t - s$ due to the Gaussian nature, i.e., $W_t - W_s \sim N(0, t - s)$. $\mathbb{E}[\mathrm{W_t}] = 0$ means that on average Gaussian motion neither drifts up nor down, and $Var(W_t - W_s) = t - s$ means that variance scales linearly with time.

Nowhere differentiable path: One of the most surprising things about Wiener process is that its path is continuous everywhere but differentiable nowhere with probability one. This jagged or fractal-like behavior is fundamental. If Brownian motion were differentiable, its derivative would represent a kind of smooth noise, which would imply local correlations, contradicting the ideal of white noise having independent values at every infinitesimal interval. The nowhere differentiability ensures that new, unpredictable randomness is introduced at every time scale.

This property is the primary reason why classical calculus is insufficient for dealing with SDEs driven by W_t. Since Brownian motion is nowhere differentiable in its path it requires Ito calculus for integration.

Markov Property: Brownian motion is a Markov process meaning that it has no memory beyond its current value. Once you know W_s at time s, everything that happened before is irrelevant for predicting W_t at any later time $t > s$. Equivalently, the distribution of future increments $W_t - W_s$ depends only on the length of the interval $t - s$, not on the detailed path the process took to get to W_s. In practice this "memoryless" property is what makes SDEs tractable: you only ever need the current state to step forward, not the entire history, which is why techniques like Euler–Maruyama can walk one increment at a time without keeping track of past values.

Martingale Property: It is also important to note that Brownian motion besides being denoted as a white noise process is also a martingale. This means that its expected future value, given the present and past, is simply the present value. In more precise terms, if W_t is a Wiener process adapted to the filtration $\{\mathcal{F}_t\}$ (which is everything we have observed up to time t), then, for any $0 \leq s \leq t$, the increment $W_t - W_s$ is independent of $\mathcal{F}_s$ and has mean zero. Accordingly,

$$\mathbb{E}[W_t | \mathcal{F}_s] = \mathbb{E}[W_s + (W_t - W_s) | \mathcal{F}_s] = W_s + \mathbb{E}[W_t - W_s | \mathcal{F}_s] = W_s$$

which is exactly the martingale property.

Quadratic variation: Despite its continuous paths, Brownian motion is so irregular that it has infinite variation over any finite interval, that is, the sum of absolute increments blows up to infinity as you make your time steps finer and finer. However, its quadratic variation over an interval $[0, t]$ is finite and precisely equal to t:

$$\langle W_t \rangle = \lim_{\|\Pi\| \to 0} \sum_i \left(W_{t_{i+1}} - W_{t_i}\right)^2 = t$$

This property, often compactly written as $(dW_t)^2 = dt$, is fundamental to the development of Itô calculus and therefore Ito Lemma which acquires an extra drift term, an effect absent under ordinary calculus.

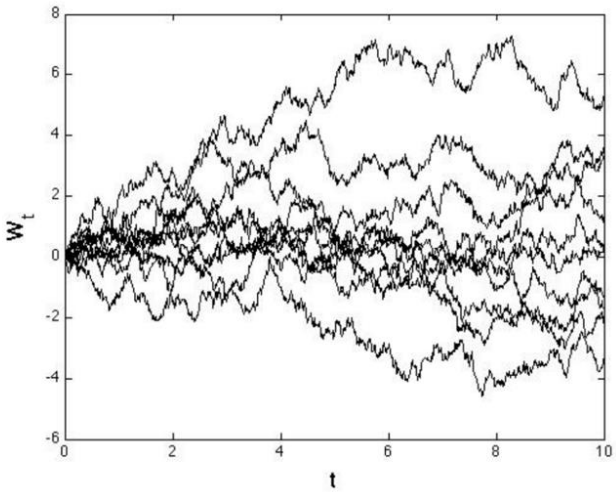


Figure 1: A sample of 10 independent Brownian motions.
Source: see Papanicolaou, A. (2019)

Itô Calculus: The Language of Stochastic Dynamics

As I already mentioned, the non-differentiable nature of Wiener process paths necessitates a specialized form of calculus, known as Itô calculus, for defining integrals with respect to W_t and for analyzing functions of stochastic processes driven by W_t.
Classical Riemann-Stieltjes integration theory requires the integrator to be of bounded variation. However, Wiener process paths have unbounded variation, rendering standard integration techniques inapplicable.

It is indeed not possible to define $\int_0^T f_t \, dW_t$ by standard Riemann–Stieltjes methods and must instead turn to *Itô's stochastic integral*, which is built around mean-square limits of simple approximations and exploits the special quadratic-variation behavior $(dW_t)^2 = dt$.
At its core, the Itô integral allows one to "add up" a stochastic process H_t against the random wiggles of a Wiener process W_t. To make sense of

$$\int_0^T H_t \, dW_t$$

we require that H_t be adapted to the noise up to time t, in other words, H_t can only depend on the past history of W, not on any future surprises. The integral is then approximated by slicing $[0, T]$ into very small intervals $[t_i, t_{i+1}]$ forming the Riemann-sum–like expression

$$\sum_{i=0}^{n-1} H_{t_i}(W_{t_{i+1}} - W_{t_i})$$

Crucially, the integrand Ht is evaluated at the left endpoint t_i of each subinterval $[t_i, t_{i+1}]$. This choice of sampling point distinguishes the Itô construction from other stochastic integrals and

underlies its characteristic properties such as the Itô isometry and the form of Itô's lemma. Convergence occurs in the mean-square (L^2) sense, which facilitates powerful variance estimates.

Key properties:

- *Linearity*: $\int_0^T (aH_t + bG_t)\, dW_t = a\int_0^T H_t\, dW_t + b\int_0^T G_t\, dW_t$
  For any adapted processes H and G and constants a, b.
- *Zero Mean*: If H_t is an adapted process satisfying the square integrability condition $\mathbb{E}[\int_0^T H_t^2 dt] < \infty$, then $\mathbb{E}[\int_0^T H_t\, dW_t] = 0$
- *Itô isometry*: $\mathbb{E}[\int_0^T (H_t dW_t)^2] = \mathbb{E}[\int_0^T H_t^2 dt]$ This vital property relates the variance of the stochastic integral to the expected integral of the squared integrand. It is fundamental for many calculations in SDE theory and for defining the L^2 convergence of the integral.
- *Martingale property*: Under suitable conditions on H_t (e.g., H_t is adapted and $E < \infty$) the process $M_t = \int_0^t H_s\, dW_s$ is a martingale with respect to $\mathcal{F}_t$.
- *Quadratic variation*: $(dWt)^2 = dt$ or equivalently, $[W]_t = t$

Each of these properties plays a crucial role in establishing existence, uniqueness, and stability for solutions of stochastic differential equations under the Itô framework.

What is Itô lemma?

Itô's Lemma, often referred to as the stochastic analogue of the chain rule in classical calculus. It provides a systematic way to compute the differential of functions of stochastic processes.
In ordinary calculus if we want to find the variation of $f(x)$ when x changes by a small quantity dx we often apply the formula $df(x) \approx f'(x)dx$. However in stochastic calculus dx includes random noise increments from Brownian motion $(dW_t)^2$, hence if in ordinary calculus small increments $(dx)^2$ are negligible, in stochastic calculus $(dW_t)^2$ is not infinitesimal and on average behaves like dt which creates an extra correction term.
If X_t follows an SDE and you want to know how a function $f(x)$ changes over time, we need to include both the first derivative and a second derivative term to capture the random variation. That's exactly what Ito lemma does.
For instance, if your asset price S_t follows a Geometric Brownian Motion:

$$dS_t = \mu S_t\, dt + \sigma S_t\, dW_t$$

And you take $f(S_t) = \ln S_t$, Ito lemma adds an extra $-\frac{1}{2}\sigma^2 dt$ in the drift of lnS_t. This extra term directly results from the quadratic variation property. It embodies how uncertainty, represented through the diffusion coefficient σ, affects non-linear functions of the process.
Itô's Lemma is fundamental for analytically solving SDEs, performing variable transformations, and constructing numerical schemes (such as Euler-Maruyama, Milstein). It also clarifies how randomness propagates through non-linear functions enabling rigorous analysis of complex stochastic systems.

Stratonovich Integral: An Alternative Formulation of the Stochastic Integral

An alternative to the Itô integral is the Stratonovich integral, which differs primarily in the evaluation point of the integrand H_t. Specifically, the Stratonovich integral evaluates H_t at the midpoint of each integration interval: $H_{\frac{t_i+t_{i+1}}{2}}$
A fundamental distinction between Itô and Stratonovich integrals lies in their calculus rules. While Itô calculus introduces an additional second-order term due to the quadratic variation of Brownian motion, Stratonovich calculus follows the classical chain rule of ordinary calculus. This property makes them particularly attractive in certain physical contexts where classical calculus intuitions are beneficial.
However, Stratonovich integrals typically require stronger smoothness conditions on the integrand and do not generally produce martingales. Nonetheless, when sufficient regularity conditions are met, conversion formulas allow for straightforward transformations between Itô and Stratonovich interpretations.
In the context of Neural SDEs, although the choice between those two integrals is occasionally discussed, the Itô formulation is predominantly adopted, especially when drift and diffusion functions are modeled with neural networks. This preference arises from the martingale properties inherent to the Itô integral and its explicit alignment with the non-anticipative nature of information flow in sequential data analysis, crucial in machine learning frameworks.

In financial modeling, the primary objective is not to reconstruct perfectly smooth paths, but to accurately capture the underlying trend and volatility structure of asset prices or returns. The Itô integral provides a natural framework for this, as it aligns with the non-anticipative nature of financial markets and preserves the martingale properties that are fundamental to no-arbitrage pricing theory. Moreover, its variance-preserving property through the Itô isometry makes it particularly well-suited for modeling and learning the evolution of volatility. While Stratonovich integration can produce smoother trajectories, this is generally less critical in financial

forecasting, where noise is a fundamental part. Consequently, the Itô formulation offers theoretical alignment and practical advantages for learning financial time series with Neural SDEs.

1.3 Neural Stochastic Differential Equations: Formulation and Core Properties

As already mentioned, Neural Stochastic Differential Equations (Neural SDEs) represent the stochastic extension of Neural ODEs. This framework leverages the expressive power of deep learning to model noisy continuous-time processes.

Formally, a Neural SDE defines the dynamics of a latent state $z_t \in R^{dz_t}$ over time through an Itô stochastic differential equation of the form:

$$dz_t = f(t, z_t;\ \theta_t)dt + g\big(t, z_t; \theta_g\big)dW_t$$

The latent process begins with an initial state z_0, which is typically computed by a neural network $h(x; \theta_h)$ that maps an input observation x (such as initial time series values or a context vector) to the latent space. The dynamics of z_t are governed by two key components, as seen in the classic formulation: the drift function $f(t, z_t; \theta_f)$, which captures the rate of change over an infinitesimal time interval dt, in the absence of noise; and the diffusion function $g(t, z_t; \theta_g)$ which determines the magnitude of stochastic fluctuations introduced by the Wiener process W_t. Both functions are parameterized by neural networks. If W_t is a dW dimensional standard Wiener process with independent components, then g becomes a matrix of shape $dz \times dW$, where dz is the dimensionality of the latent state. The full set of learnable parameters $\theta = \{\theta_f, \theta_g, \theta_h\}$ is optimized during training to capture the system's dynamics.

Several architectural choices could be chosen for the neural networks f, g, and h, such as Multilayer Perceptrons (MLPs), Convolutional Neural Networks (CNNs), or Recurrent Neural Networks (RNNs), depending on the specific nature of the input data x and the latent state z_t, as well as the requirements of the task.

The choice of how to parameterize the diffusion function g, as a scalar, a diagonal matrix, or a full matrix, and whether it depends on the state z_t, time t, or both, significantly influences both the complexity of the stochastic process that can be learned and the practical difficulty of training the model. A simpler structure for g, such as state-independent additive noise, where g is constant, or diagonal multiplicative noise, might lead to more stable training and easier satisfaction of theoretical conditions, but at the cost of reduced expressiveness in modeling complex, state-dependent volatility structures or correlations between noise sources. Conversely,

a full matrix $g(z_t, t; \theta_g)$ maximizes flexibility, but it also introduces substantial challenges such as difficulties in guaranteeing well-posedness, increased variability during training, and greater computational burden for numerical solvers. Balancing this trade-off is a central consideration in the design of Neural SDEs.

Existence and Uniqueness of Solutions

The well-posedness of a Neural SDE, specifically, the existence and uniqueness of its solution, constitutes a fundamental theoretical prerequisite. If your Neural SDE doesn't have a well-defined solution, if it doesn't reliably "exist" in a mathematical sense, or if it could have multiple, different solutions for the same starting point, then any model's predictions rest on an unstable foundation. The established theory for classical SDEs provides the necessary conditions for ensuring well-posedness.

The existence of a unique strong solution is guaranteed if the drift coefficient $f(t, x)$ and the diffusion coefficient $g(t, x)$ satisfy two primary conditions globally with respect to the state x and uniformly in time t over any finite interval $[0, T]$.

- *Lipschitz continuity*: the drift and diffusion can't be overly sensitive to tiny changes in the system's state. Think of it like this: if your system is in state A, and then moves to a very slightly different state B, the underlying trend (f) and randomness (g) shouldn't suddenly become wildly different. There's a limit (a constant L) to how much they can change relative to how much the state itself changed:

  $$\|f(t, x) - f(t, y)\| + \|g(t, x) - g(t, y)\| \leq L\|x - y\|$$

  This rule keeps the system's behavior smooth and predictable, preventing solutions from veering off track due to tiny disturbances.

- *Linear Growth Condition:* As the system's state (x) gets very large, the influence of the drift and diffusion can't explode even faster. Their growth is capped by a constant C relative to the size of the state:

  $$\|f(t, x)\|^2 + \|g(t, x)\|^2 \leq C(1 + \|x\|^2)$$

  This condition restricts the rate at which the magnitudes of the coefficients can increase as $\| x \| \to \infty$, ensuring that the solution does not explode to infinity in finite time.

In the context of Neural SDEs, its networks are incredibly flexible, which is great for capturing complex patterns. But this flexibility is a double-edged sword.

Standard neural networks, with common ingredients like Rectified Linear Unit (ReLU) activation functions, don't inherently obey the "don't change too fast" or "don't grow too wildly" rules. A ReLU, for example, isn't globally smooth in the way the Lipschitz condition requires.
Consequently, specific interventions may be necessary during the training phase or in the architectural specification to promote or enforce these critical properties. We can gently guide the neural network during its learning process using common methodologies such as globally Lipschitz activation functions. While classical examples include *tanh* and *sigmoid*, more recent developments have introduced specifically designed Lipschitz activations like *LipSwish*, which aims to combine the benefits of Swish-like activations with guaranteed Lipschitz continuity. Techniques such as *weight clipping* or, more effectively, *spectral normalization* of weight matrices can control the Lipschitz constant of individual layers and, by extension, the overall behavior of the network. An alternative or complementary approach involves directly penalizing deviations from desired gradient properties. *Gradient penalty* terms can be added to the loss function, explicitly discouraging large gradients of the network's output with respect to its input. This approach, notably used for stabilizing GAN training by enforcing Lipschitz constraints, can also serve to promote better-behaved drift and diffusion functions in Neural SDEs.

Or, instead of letting the neural networks for drift and diffusion be completely free spirits, some researchers design them with specific structures that are *guaranteed* to be stable. For example, Langevin-type SDEs or Linear Noise SDEs. These might be a bit less flexible overall, but they come with the peace of mind that their solutions will exist and be unique.

This is a core tension in Neural SDE research: we want maximum expressivity (the ability to learn almost anything) from our neural networks, but we also need mathematical stability. Finding the right balance is key. Clearly, making sure our Neural SDE is well-posed is fundamental to building trustworthy and useful models.

Universal Approximation Capabilities

A cornerstone of Neural SDE theory is understanding their capacity to approximate a vast array of stochastic processes. This is vital, as it determines whether Neural SDEs can truly learn to represent the diverse and complex real-world systems we encounter. The answer lies in specialized Universal Approximation Theorems (UATs) developed for this unique context.
Classical universal approximation theorems (UATs) are insufficient for Neural SDEs because they don't control the global growth of the neural networks used for drift and diffusion, potentially leading to ill-posed SDEs with exploding solutions (see Kwossek et al. on *Universal*

approximation property of neural stochastic differential equations). The "universal approximation property under a linear growth constraint" directly addresses this critical issue. This property ensures that for any target continuous function (representing either drift or diffusion) that itself exhibits at most global linear growth, an approximating neural network can be found which achieves two key goals: first, it can approximate the target function with arbitrary accuracy uniformly on any specified compact (bounded) set. Second, and most importantly, this approximating neural network must itself globally satisfy a linear growth condition, where its growth constant does not depend on the desired accuracy or the chosen set. This dual capability of accurate approximation combined with rigorously controlled growth is fundamental, as it allows Neural SDEs to reliably approximate the solutions of true SDEs whose coefficients are well-behaved without the Neural SDE model itself becoming unstable.

Numerical Methods for Simulating Neural SDEs

Since analytical solutions to SDEs are rarely available, especially when their coefficients are complex functions like neural networks, numerical approximation schemes are essential for simulating trajectories from Neural SDEs.
The core challenge in simulating SDEs lies in appropriately handling the stochastic (noise) term. Unlike ordinary differential equations (ODEs), where standard numerical solvers can be readily applied, SDEs require methods that correctly approximate the statistical properties of the noise.

The choice of numerical solver profoundly impacts the learning process. A solver's accuracy and stability dictate how faithfully simulated trajectories represent the true solution. Poor approximations lead to inaccurate loss evaluations and gradients, potentially misguiding optimization. Unstable solvers can cause exploding trajectories and NaN values, stopping training.

- *Baseline solver: Euler-Maruyama Method*

The most fundamental and widely used method is the Euler-Maruyama (EM) scheme. Its appeal lies in its simplicity and direct extension of the familiar Euler method for ODEs. Consider a Neural SDE describing the evolution of a latent state z_t, the EM method discretizes time into small steps of the same length, denoted as $\Delta t = (T - t_0)/N$, such that $t_{k+1} = t_k + \Delta t$.
For each step $k = 0, 1, \dots, N - 1$ the approximation of the next time step z_{k+1} is computed from the current approximation z_k as:

$$z_{k+1} = z_k + f(z_k, t_k)\Delta t + g(z_k, t_k)\Delta W_k$$

Here, ΔW_k is a random number drawn from a Gaussian distribution with a mean of 0 and a variance of Δt.

The intuition behind this method is that at each time step, we advance the system based on the deterministic trend (f) and then introduce a random perturbation. The magnitude of this “kick" is determined by the diffusion term (g) and a random draw from the noise process, effectively capturing the continuous influence of random factors on the system's trajectory.

The EM method, despite its straightforward nature, is valuable for revealing crucial aspects of neural behavior. Its utility is defined by its convergence. Because its strong convergence is not very fast, achieving high accuracy for individual paths requires small time steps, which can significantly increase the total computation time. However, the method can struggle and may become unstable, causing convergence issues, when dealing with systems whose behavior changes very rapidly. These settings require careful tuning**.**

- *Higher-Order Methods*

While the Euler-Maruyama method provides the bedrock for numerical SDE approximation, it can present some limitations, particularly in terms of stability for certain types of SDEs. This has led to the development of higher-order methods, such as the *Milstein scheme* and various *Runge-Kutta* type schemes, like the *Heun method*.

These more sophisticated methods typically involve adding further terms from the Itô–Taylor expansion (the stochastic equivalent of a Taylor series), which includes multiple stochastic integrals of increasing order and requires repeated evaluations of the drift and diffusion functions at each time step.

The *Milstein scheme*, for instance, adds a correction term that improves the strong order of convergence to 1.0 for SDEs with commutative noise. This means it generally provides more accurate sample paths for a given step size Δt.

However, the implementation of these higher-order methods can be significantly more complex, especially for systems of SDEs or SDEs with non-commutative noise where additional Lévy areas (second-order integrals) may be required to maintain the order of convergence.

Together, these theoretical and mathematical tools provide the foundation upon which the Neural SDE model architecture developed in this thesis is built.

Chapter 2: Literature Review

While the previous chapter outlined the theoretical principles behind Neural Stochastic Differential Equations, this section turns to their practical deployment, specifically, their use in modeling, simulating and understanding financial time series. The aim is to position the present research within the existing body of work, highlighting how Neural SDEs have been applied to capture the stochastic, nonlinear, and temporal complexities inherent in financial markets.
This chapter critically discusses the methodological choices and performance outcomes reported in the literature and contrasts them with the specific approach taken in this thesis.

2.1 Evolution of Stochastic Modeling in Finance: From Classical to Neural Approaches

Stochastic Differential Equations have traditionally served as the cornerstone for modeling continuous-time stochastic processes in finance. Their ability to describe the infinitesimal evolution of asset prices, interest rates, and volatility has made them indispensable tools for derivative pricing, risk management, and portfolio optimization.

The literature presents a rich family of SDE-based models tailored to specific financial phenomena. Among the most foundational is the Geometric Brownian Motion (GBM), which, despite its simplicity, underpins the well-known Black-Scholes-Merton option pricing model (Black & Scholes, (1973); Merton, (1973)). For interest rate dynamics, models such as those proposed by Vasicek (1977) and Cox, Ingersoll, & Ross (1985) introduced mean-reverting SDEs. To address the empirical observation of non-constant volatility (the "volatility smile"), stochastic volatility frameworks like the Heston model (1993) were developed, where volatility itself is driven by a separate SDE, correlated with the asset price process. These models have been extensively used for pricing exotic derivatives, hedging, and simulating future market scenarios.
Despite their widespread adoption, those models rely on pre-specified functional forms and often make strong assumptions, such as log-normality of returns, constant coefficients, or specific mean-reversion patterns, that may not adequately capture the full spectrum of financial market behavior.

The literature frequently points out several limitations:

- Many financial time series exhibit well-documented "stylized facts" such as heavy tails in return distributions, volatility clustering, leverage effects, and long-range dependence, which are often difficult to incorporate parsimoniously into traditional SDEs (Cont, 2001).

- Real-world financial dynamics are often highly non-linear, and pre-defining the correct non-linear structure in an SDE can be challenging and prone to misspecification.
- Financial markets can undergo structural breaks or regime shifts, which fixed-parameter SDEs struggle to adapt to dynamically.
- Calibrating complex, multi-factor SDE models to market data can be computationally intensive and sometimes leads to unstable parameter estimates.

These limitations have motivated the exploration of more flexible modeling paradigms.
The desire to combine the continuous-time modeling strengths of differential equations with the powerful function approximation capabilities of machine learning techniques, led to the development of Neural Differential Equations.

One notable aspect in the development of neural networks for differential equations is the emergence of distinct philosophical viewpoints regarding their role. The first, often termed the "*neural differential equation*" approach, and which is the focus of this thesis, views neural networks as generative models or solution operators. In this context, the goal is to learn a system, like an SDE-GAN, that can produce new sample paths characteristic of the SDE underlying the training data. The second, the "*universal differential equation*" approach, pioneered by Rackauckas et al. (2020), employs neural networks to discover unknown functional components within a predefined dynamical system structure. Here neural networks are used to model these identified gaps, such as specific influences on the drift or diffusion, rather than defining the entire dynamics from scratch. This distinction is important as it implies different modeling goals and methodologies in the construction and application of Neural SDEs.

The conceptual journey began with the introduction of Neural Ordinary Differential Equations (Neural ODEs) by Chen et al. (2018). This work established the foundations of learning continuous-time dynamics by parameterizing the derivative of a system's hidden state with a neural network. A key innovation was the application of the adjoint sensitivity method for gradient computation, which allowed for memory-efficient training of these continuous-depth models. This new method was subsequently adapted for Neural SDEs.

Several key publications have laid the groundwork and driven advancements in the Neural SDE field, leading to important architectural developments aimed at improving stability, expressiveness and applicability.

Tzen & Raginsky (2019) and subsequently Li et al. (2020) made early inroads into Latent variable Neural SDEs. They popularized the concept of using an encoder to map observations to an initial latent space, which evolves accordingly to a learned SDE. A decoder can then map the latent path back to observation space. They provided a way to model complex distributions evolving stochastically, with applications including long-term portfolio dynamics and modeling of latent volatility factors in derivative pricing.

Liu et al. (2019) introduced Neural SDEs with built-in regularization, incorporating techniques like dropout and noise injection directly into the training process. These mechanisms help prevent overfitting and improve the model's ability to generalize in noisy environments. They also proposed a specific variant, the Neural Linear Noise SDE, which enforces linearity in the diffusion term, with a focus on improving model stability.

Recognizing that naïve parametrization of drift and diffusion can lead to instability or ill-defined SDEs, Oh et al. (2024) proposed specific classes of Stable Neural SDEs. These include the Neural Langevin-type SDE (LSDE), Neural Linear Noise SDE (LNSDE), and Neural Geometric SDE (GSDE), each with structures designed to ensure the desirable theoretical properties such as existence, uniqueness and stability, making them more robust, also in financial contexts where stability and theoretical guarantees are crucial for risk management and regulatory applications.

Another notable contribution comes from the development of Neural Controlled Differential Equations (Neural CDEs) (Kidger et al., 2020), which extend continuous-time modeling to irregularly sampled sequences using control theory. While distinct from NSDEs, Neural CDEs are conceptually related and are able to learn functions of controlled paths, offering an alternative and flexible way of processing sequential information. In this thesis, a Neural CDE is used as the discriminator in the adversarial training setup, enabling it to evaluate generated trajectories as full continuous paths rather than fixed-length vectors. This pathwise perspective makes CDEs especially suitable for detecting subtle mismatches between real and synthetic time series.

To capture sudden, discontinuous movements often observed in financial markets, some researchers have extended NSDEs to Neural Jump SDEs (e.g., Jia & Benson (2019)). These models incorporate jump processes, like Poisson processes, into the dynamics, allowing for more realistic modeling of event driven financial data, such as credit defaults or macroeconomic announcements. Zhang et al. (2024) extended this to temporal point processes.
This modeling strategy can be framed within the broader theory of Lévy-driven SDEs (Applebaum, D. (2009)), where jumps arise from a Lévy process rather than purely Gaussian

noise. In this context, Poisson processes represent a specific case, and more general Lévy processes could offer enhanced flexibility in capturing heavy-tailed, asymmetric, or clustered jumps often seen in financial data. Although this added complexity requires specialized numerical solvers, it provides a principled approach to effectively capture rare, discontinuous events with NSDEs.

2.2 Generative Adversarial Networks (GANs) for Sequential Data

While Neural SDEs offer a powerful way to model continuous-time stochastic processes, the framework chosen in this research for their simulation, Generative Adversarial Networks (GANs), has its own rich history, particularly in adapting to the complexities of sequential data like financial time series.

The concept of GANs, introduced by Goodfellow et al. (2014), marked a significant paradigm shift in generative modeling. Their work proposed a framework where two neural networks, a generator and a discriminator, are trained in an adversarial process. This approach demonstrated a powerful ability to learn complex data distributions, initially with impressive success in image generation, and quickly motivated research into its application across diverse data types.

Researchers have extensively documented the difficulties in directly applying standard GAN architecture to time series. Key issues highlighted include the robust modeling of long-range temporal correlations, handling non-stationarities often present in financial data, and effectively managing sequences of variable lengths (Esteban, Hyland, & Rätsch, 2017; Yoon, Jarrett, & van der Schaar, 2019).

To address these challenges, the literature presents several architectural adaptations.

Early influential works focused on integrating Recurrent Neural Networks (RNNs), particularly LSTMs and GRUs, into GAN components due to their inherent ability to process sequences. Examples include C-RNN-GAN by Mogren (2016) for continuous sequential data and the Recurrent Conditional GAN (RCGAN) framework explored by Esteban, Hyland, & Rätsch (2017) for medical time series.

Yoon, Jarrett, & van der Schaar (2019) introduced TimeGAN, a framework specifically developed to capture the temporal dynamics in time series. Their approach combined unsupervised adversarial training with a supervised loss related to step-wise predictions. TimeGAN has since became a frequently cited benchmark.

More recent literature has explored the incorporation of attention mechanisms and Transformer networks into GANs for time series and these architectures are often investigated for their potential to capture long range dependencies compared to traditional RNNs.

An important refinement to the original GAN framework was introduced by Arjovsky et al. (2017), who proposed the Wasserstein GAN (WGAN). WGANs replace the standard Jensen-Shannon divergence with the Wasserstein distance, leading to more stable gradients and improved training dynamics. However, enforcing the required Lipschitz continuity of the discriminator posed practical challenges. This was addressed in WGAN-GP by Gulrajani et al. (2017), who introduced a gradient penalty term that regularizes the norm of the discriminator's gradients. Gradient penalty has since become a standard stabilization technique across many time series GAN applications. Notably, Kidger et al. (2021) incorporated it into their SDE-GAN framework, where it plays a crucial role.

The ongoing development of GANs for sequential data provides important context for understanding the choices and benefits of integrating them with Neural SDEs for financial time series simulation.

2.3 Neural SDEs: Training Paradigms and Integration into GANs

Effective training methodologies are fundamental to unlocking the potential of Neural SDEs, especially for complex applications like stock prices modeling. The literature presents a spectrum of approaches for fitting these models to data.

Training NSDEs, as previously outlined, involves optimizing parameters θ_f, θ_g of the NN representing drift and diffusion terms, typically by minimizing a loss function that measures the discrepancy between the generated paths and real data, or by maximizing likelihood.

- *Likelihood-Based Approaches*

  For SDEs where the transition density (i.e., the conditional probability of the system's future state) is tractable or can be approximated, MLE is a standard approach. Some Neural SDE literature explores maximizing the likelihood of observed data paths under the learned SDE model. For instance, Shen & Cheng (2025), in their work reportedly use a maximum likelihood objective, coupled with simulation-free training schemes.

  When NSDEs are used as latent variable models, Variational Inference is a common training paradigm. Tzen & Raginsky (2019) employed this for their Latent SDEs, optimizing an

Evidence Lower Bound (ELBO) that consists of a reconstruction term (how well data is explained by latent paths) and a Kullback-Leibler (KL) divergence term that regularizes the latent path distribution.

- *Simulation-Based and Moment-Matching Techniques*

  Simulation-based methods were influential in early SDEs fitting, and adapted for some Neural SDE methodologies, involve generating paths from the model and aligning their statistical properties with observed data. The "universal differential equation" framework, as discussed by Rackauckas et al. (2020), notably employs such techniques. Within this framework moment-matching losses are often utilized, where model parameters are tuned by comparing summary statistics, such as mean and variances of simulated paths against those from real data. (e.g., Vaisband et al. (2023), build upon this work).

- *Simulation-Free and Score-Matching Inspired Techniques*

  To avoid costly path simulations, some research focuses on "simulation-free" objectives. Vaisband et al. (2023), building on SDE coefficient inference, explore various loss formulations including those based on pseudo-likelihoods derived from SDE discretization schemes (e.g., Euler-Maruyama or Lie-Trotter splitting based losses), offering alternatives to direct simulation.

  Inspired by score-based generative modeling, Song et al. (2021) showed how SDEs can be defined to match data scores, though this often involves running a corresponding reverse-time SDE. The work on SDE Matching (e.g., Bartosh et al. (2025)) aims for more direct, simulation-free training of latent SDEs by matching the learned SDE's properties to those derived from data.

- *Advanced Expectation Approximation*

  When loss functions involve expectations over path space, standard Monte Carlo methods are typically used to approximate these expectations by averaging over simulated sample paths. However, this can be computationally expensive. Snow & Krishnamurthy (2025) propose using Wiener-space cubature methods to approximate these expectations with a weighted sum of deterministic ODE solutions, potentially offering faster convergence rates.

The successful training of any Neural SDE relies heavily on two aspects widely discussed in the literature. The selection of an appropriate solver, which can range from foundational schemes like Euler-Maruyama and Milstein to other options such as the Reversible Heun method

proposed by Kidger et al. (2021), impacts both the accuracy of the forward simulation and the properties of the backward pass for gradients.
Complementing the solver, efficient gradient estimation is paramount; the adjoint sensitivity method, generalized to SDEs by Li et al. (2020), serves as a cornerstone for memory-efficient and scalable gradient computation.

- *Neural SDE-GANs*

The integration of Neural SDEs as generators within Generative Adversarial Network frameworks (SDE-GANs) synergistically combines the continuous-time stochastic modeling strengths of NSDEs with the powerful distributional learning capabilities of GANs. Critically, GANs, despite their noted training challenges, are widely recognized for their ability to produce exceptionally sharp and realistic samples that often capture subtle details in complex data distributions. This success is partly because they learn implicitly by discriminating between real and generated data bypassing the need to define or optimize an explicit likelihood function. For complex models like Neural SDEs, such likelihood functions are often found to be analytically intractable and, even when approximated, can lead to inaccuracies or computational challenges in the estimation process.

The seminal work by Kidger et al. (2021) formally established this SDE-GAN paradigm, where a Neural SDE generates continuous sample paths from noise, and a discriminator learns to distinguish these paths from real data trajectories, effectively viewing the generation process through the lens of SDEs operating in path space.
This framework has proven particularly promising in the context of financial modeling, where it enables the generation of synthetic asset price trajectories that preserve the complex stochastic properties of real markets.

While this foundational SDE-GAN literature employs Stratonovich SDEs, this thesis adopts the Itô formulation for the Neural SDE generator. The choice is motivated by the desire to maintain consistency with the classical interpretation of SDEs in finance, particularly regarding non-anticipating integrands and martingale properties, even if it necessitates careful handling of the Itô-Stratonovich conversion for certain numerical methods or theoretical comparisons. The general path-space GAN framework established by Kidger et al. (2021) serves as a key conceptual foundation for the Neural SDE-GAN developed and explored in this thesis, yet with the aforementioned modifications and a specific focus on financial time series simulation.

Training these SDE-GANs typically involves an adversarial loss function, often derived from min-max or Wasserstein GAN principles, applied to features of the entire paths.

The well-documented challenges of GAN training necessitate careful architectural design, loss function selection, and regularization, alongside efficient gradient computation for the NSDE generator leveraging the adjoint sensitivity method (Li et al., 2020; Kidger et al., 2021).

Recent works based on rough path theory have proposed using path signatures as compact descriptors of trajectory geometry. Ni et al. (2021) introduced SigWGAN, which applies the signature transform (a mathematical summary of the path's geometry) as input to the discriminator, combined with the Wasserstein-1 distance to compare paths with respect to their temporal ordering. This approach reframes adversarial training as a supervised learning problem. Liao et al. (2023) extended this approach to conditional generation, enabling trajectory synthesis based on external variables. While these methods offer theoretical advantages, computing signature transforms, particularly in high dimensions or over long horizons, introduces substantial computational overhead. Given these constraints, this thesis adopts a standard WGAN-GP framework, leaving signature-based extensions as a direction for future work.

Chapter 3: Methodology

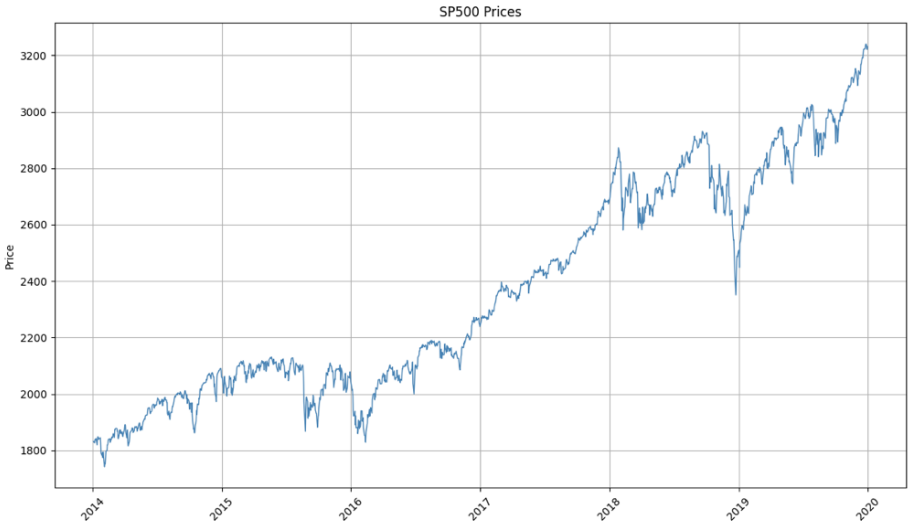
This chapter presents the methodology for designing and training a generative model of financial time series based on a Neural Stochastic Differential Equation GAN (SDE-GAN).
This approach allows to learn the path-level distribution of stock prices, treating the generator as a Neural SDE that maps Brownian noise into trajectories, and the discriminator as a neural controlled differential equation (CDE) that assesses the realism of these paths.

3.1 Data Acquisition and Preprocessing

Data Source and Description

The dataset used in this study consists of closing prices of the S&P 500 index. The data spans from January 1, 2014, to December 31, 2019, providing six years of market activity under varying economic conditions.
The source of the data is *Investing.com* and the frequency is daily. Only trading days are included, excluding weekends and market holidays.

The S&P 500 index was chosen for this study as the target financial time series due to its relevance, liquidity, and representativeness of the broader U.S. equity market. As one of the most widely followed financial benchmarks, it includes 500 of the largest publicly traded U.S. companies, the S&P 500 reflects aggregate market behavior and investor sentiment, making it a standard reference index in both academic research and industry practice. It has rich dynamics such as volatility clustering and long-range dependencies.



	Close
count	1510.000000
mean	2365.649801
std	381.180558
min	1741.900000
25%	2050.725000
50%	2265.200000
75%	2723.100000
max	3240.000000

Figure 2: Evolution of S&P 500 Closing Prices from 2014 to 2019 with Summary Statistics

The dataset consists of 1510 daily closing prices ranging from 1741.9 to 3240.0, with a mean of 2365.6 and a standard deviation of 381.2.

The time period from January 2014 to December 2019 was deliberately chosen for several reasons:

1. *Data quality and stationarity:* This interval avoids the extreme volatility introduced by the COVID-19 pandemic starting in early 2020, which could otherwise dominate the learned dynamics. The selected window is relatively stable, allowing the model to capture structural patterns of "normal" market behavior without being overwhelmed by rare shock events.

2. *Length and granularity:* With daily closing prices over six years, the dataset contains sufficient observations to train deep generative models while maintaining temporal resolution appropriate for modeling medium-term financial trends.

3. *Market diversity:* The selected period spans several market regimes including moderate bull phases and short-term corrections, ensuring that the model is exposed to a variety of dynamics without being biased toward any single phase.

By focusing on this specific dataset and time range, the aim is to train the model on patterns that are both economically meaningful and statistically rich, enabling the generator to synthesize trajectories that align with the real-world behavior of financial assets under typical conditions.

Initial Data Transformation

Logarithmic returns (log-returns) were used as the primary input for training the model. Compared to raw prices or log-prices, log-returns offer several advantages for generative modeling: they are stationary, naturally centred around zero, and scale-independent, which makes them particularly suitable for learning the underlying dynamics of financial time series. Preliminary experiments also showed that models trained directly on log-prices struggled to capture realistic path behavior and tended to suffer from instability.

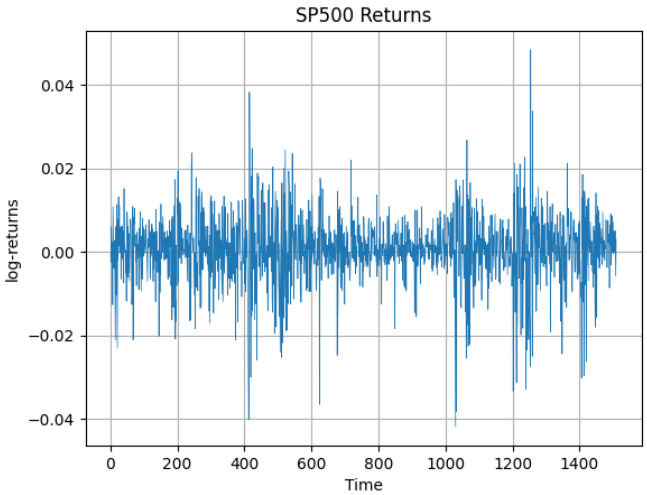


Figure 3: Daily Log-Returns of the S&P 500 Index (2014–2019)

While financial datasets can sometimes contain missing values, the data employed for this research was found to be complete, obviating the need for imputation. No additional corporate-

action adjustments were necessary because the data source provided *adjusted closing prices*, which already incorporate both stock splits and dividends.

Distribution Analysis

```
Excess Kurtosis: 3.6976042478582816
Skewness: -0.5196022344348082
```

Skewness of -0.52 indicates slight left skew, meaning that negative returns are more frequent or more extreme than positive ones.

Excess kurtosis value is above the normal distribution's value of 3. High excess kurtosis indicates a leptokurtic distribution, meaning heavier tails than a Gaussian distribution and higher probability of extreme values. This property is typical of financial returns and underscores the importance of capturing tail risk and volatility clustering.

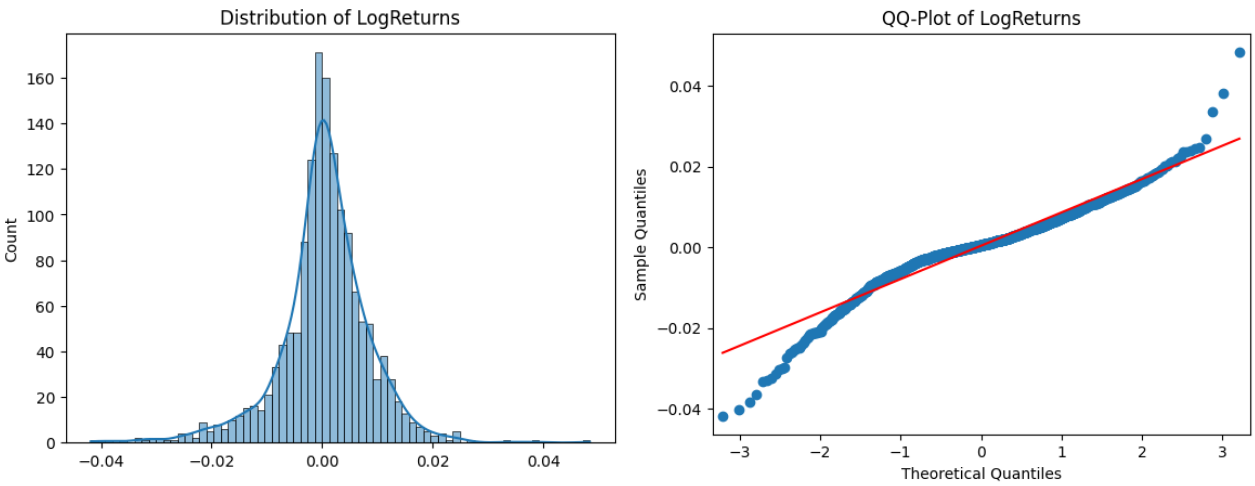


Figure 4: Histogram and Q-Q Plot of S&P 500 Log-Returns Distributions

The histogram (*Figure 4*) shows a peaked, narrow center with fat tails, meaning that most returns are very close to 0, but there are more extreme values (both positive and negative) than a Gaussian would predict. The Q-Q plot (*Figure 4*) compares the quantiles of the data to those of a normal distribution, the curved S-shape shows heavy tails and the points far from the red line at both ends indicate extreme outliers as expected and confirmed by kurtosis and skeweness. These statistical features justify the choice of a Neural SDE framework with learnable diffusion.

Outlier Handling

Financial returns are known to exhibit heavy tails and occasional extreme values, which are genuine reflections of market shocks rather than noisy errors. For this reason, no outlier removal

was performed. To prevent these extreme values from skewing the learning dynamics, a scaler robust to outliers was selected.

Data Scaling

Neural networks are sensitive to the scale of their input features. To ensure stable and efficient training, the log-return values were scaled using *RobustScaler* from *sklearn.preprocessing* library, which centres the data using the median and scales it according to the interquartile range (IQR). This choice was motivated by the non-Gaussian, heavy-tailed nature of return distributions, where standard scaling methods like *StandardScaler* (based on mean and standard deviation) would be overly influenced by outliers.
The scaler was fitted exclusively on the training data to avoid information leakage from the test set. Once fit, it was applied to transform both the training and test data consistently.

Sequence Generation

To model the sequential structure of financial time series, the full time series was segmented into overlapping, fixed-length windows of 50 time-steps. Each window represents a realistic temporal segment that the model is expected to generate. This framing allows the model to learn localized dynamics while still capturing meaningful medium-term patterns.
A custom *TimeSeriesDataset* class was implemented, which efficiently slices the dataset into input sequences. Each sample provided to the model consists of a pair of tensors (time, value), enabling the model to leverage time as an explicit input channel. This setup is particularly beneficial for models based on neural differential equations, which treat time as a continuous control variable.

3.2 SDE-GAN Model Architecture

The model architecture consists of two main neural networks competing in an adversarial game: the Generator, which learns to create synthetic time series, and the Discriminator, which learns to distinguish synthetic series from real ones. Both components utilize Multi-Layer Perceptrons (MLPs) as fundamental building blocks.

The MLP class defines a standard feed-forward neural network, used as a modular component in both the Generator and the Discriminator. At its core, an MLP is a function approximator that learns to map input data to output data through a series of transformations.

It accepts an input tensor of dimension *in_size* and, after processing through its layers, produces an output tensor of dimension *out_size*.

An MLP (*Figure 5*) is composed of multiple layers of interconnected "neurons" (or units). The first layer is the input layer, the last is the output layer, and any layers in between are called hidden layers. In this implementation, the MLP consists of *num_layers* linear transformations, each followed by an activation function, except the final one.

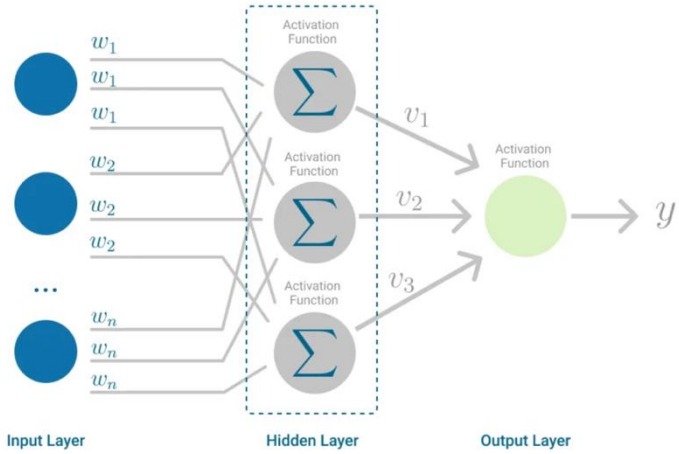


Figure 5: Schematic Representation of a Feedforward Neural Network
Source: see Bento, C. (2021)

Each linear layer performs an affine transformation on its input. If x is the input to a layer, the output y is computed as $y = Wx + b$, where W is a matrix of weights and b is a vector of biases. These weights and biases are the parameters of the network that are learned during the training process. The first linear layer maps the input from *in_size* to *mlp_size* (the size of the hidden layers). Subsequent hidden layers transform from *mlp_size* to *mlp_size*. The final linear layer maps from *mlp_size* to the specified *out_size*.

The information flows in one direction, from the input layer, through the hidden layers, to the output layer, without forming cycles. This is why it's called a "feed-forward" network.

After each linear transformation, a non-linear activation function is applied element-wise to the output. This non-linearity is crucial; without it, a multi-layer network would be mathematically equivalent to a single linear transformation, severely limiting its ability to model complex relationships.

In this architecture, the LipSwish activation function is used. LipSwish is a scaled variant of the SiLU (Sigmoid-weighted Linear Unit) function, defined as $0.909 * x * sigmoid(x)$. The SiLU function itself is known for its smooth, non-monotonic nature, which can sometimes lead to better performance than traditional activations like ReLU. The scaling factor (0.909) in LipSwish is specifically chosen to help control the Lipschitz constant of the network. This property is particularly beneficial for the discriminator in a WGAN-GP setup, as it aids in

stabilizing the adversarial training process by encouraging the discriminator to be (approximately) 1-Lipschitz.

A Tanh activation function was applied after the final linear layer. The Tanh function squashes its input to the range [-1, 1]. This is often used when the desired output of the MLP module needs to be bounded, for example, when parameterizing the drift and diffusion functions of an SDE, constraining their rate of change can improve the stability of the SDE solver.

In essence, the MLP learns a complex mapping by composing multiple simple linear transformations and non-linear activations. The depth (*num_layers*) and width (*mlp_size*) of the MLP determine its capacity to approximate intricate functions. During training, the weights and biases of these linear layers are adjusted via backpropagation to minimize a loss function, enabling the MLP to perform its designated role within the larger Generator or Discriminator architecture.

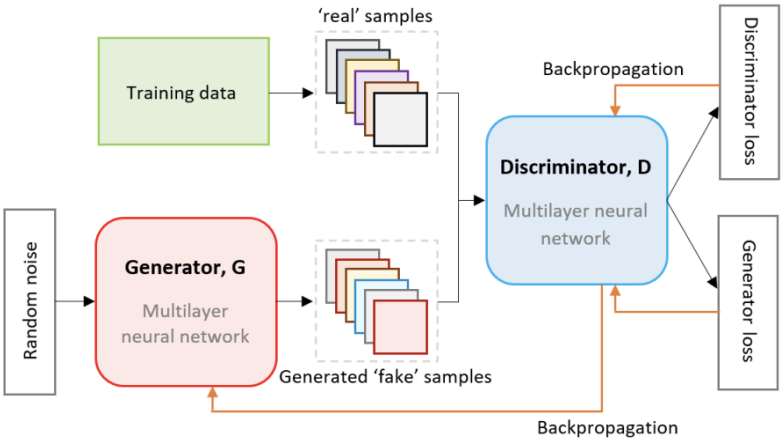


Figure 6: Example of GAN architecture
Source: see Little et al. (2021)

The entire model architecture was implemented using the *PyTorch* framework, with *torchsde* and *torchcde* libraries enabling the simulation and training of the Neural SDE generator and the CDE-based discriminator, respectively.

Generator

The Generator is designed to transform a source of randomness into realistic time series paths. It achieves this by implementing a Neural Stochastic Differential Equation (Neural SDE), where the core dynamics are learned by neural networks.

The process begins by drawing a vector of pure Gaussian noise from a standard normal distribution, this serves as the initial source of randomness for a given path, which guarantees

diversity across generated trajectories. Instead of feeding the raw noise directly into the SDE, it is first fed into the feed-forward network (MLP) that “shapes” it into a coherent starting state in the model’s latent space, the initial hidden state X_0. By omitting any final squashing (Tanh) activation, this network can freely learn the optimal, unconstrained initialization.
However, early experiments revealed a tendency for the trajectories to initiate from a common starting point around zero. To counteract this observed lack of initial diversity and explicitly encourage a wider spread of starting conditions, after a base X_0 was produced, this state was further perturbed by adding a small amount of independent Gaussian noise, specifically $\varepsilon = 0.05 \times N(0, I)$.

That structured hidden state then serves as the launch pad for the stochastic differential equation to evolve the path continuously over time. The initial MLP effectively learns an optimal mapping from a simple noise distribution to a more structured initial representation for the SDE.

At its core, the generator relies on a learned continuous-time process defined by two neural networks which together form the stochastic differential equation.

- *Drift Network*: A MLP which determines the deterministic component of the SDE's evolution. It reads in the current time t and the hidden state X_t (concatenated as input) and outputs a vector that dictates the "average" direction and speed of movement of X_t. The MLP uses LipSwish activations internally and a Tanh function on its final output, constraining the instantaneous drift contribution to the range $[-1, 1]$.
- *Diffusion Network*: Also a MLP that defines the stochastic fluctuations added at each step, and the sensitivity of X_t to the random Wiener process W_t. It also takes t and X_t as input and its output is a weight matrix that maps increments of the Wiener process into the latent space. It specifies precisely how each dimension of the Brownian motion perturbations influences each dimension of the hidden-state trajectory. The diffusion network also uses a Tanh activation function. This choice helped stabilize training by smoothly bounding the output and preventing extreme values in the diffusion term. An alternative approach using output clamping was tested to allow more flexibility in volatility, but it introduced gradient issues.

Both the clamped diffusion output and the drift network's output are then multiplied by separate learnable scaling parameters. These learnable scales empower the model to adaptively tune the magnitudes of the drift and diffusion terms during training. They were initialized at 1, after observing that starting from lower values helped the generator gradually increase them over time

without being penalized too heavily from the start. In contrast, higher initial values often caused the generator to produce flat trajectories, as it became too cautious to explore.

Forward pass

From the learned initial hidden state, the generator unfolds that state through time by numerically integrating the stochastic differential equation that governs its dynamics. Since the model follows the Itô calculus framework, the Euler–Maruyama method is employed as numerical solver for integration, the classical approach widely used for simulating Itô-type dynamics.

This scheme advances the Neural SDE forward in time, producing the hidden trajectory by iteratively updating the state based on drift and diffusion dynamics. As previously mentioned, the Euler–Maruyama method requires relatively small time-steps for numerical stability and accuracy; in this setup, a step size of Δt = 0.2 was adopted.

At each timestep t_i, the latent state X_i is updated using:

$$X_{i+1} = X_i + f(t_i, X_i)\,\Delta t + g(t_i, X_i)\,\Delta W_i$$

where $\Delta W_i \sim N(0, \Delta t)$.

By repeating this update for each consecutive pair (t_i, t_{i+1}), the solver reconstructs the full latent path $\{X_{t_0}, X_{t_1}, \dots, X_{t_N}\}$.

Immediately after the continuous-time integration completes, the generator applies a linear readout to the entire latent trajectory. Concretely, once the numerical solver has produced the sequence of hidden-state vectors, each X_t is passed through a single fully connected linear layer which converts the high-dimensional hidden trajectory into a one-dimensional time series of synthetic observations $\{y_{t_0}, y_{t_1}, \dots, y_{t_N}\}$, which are, in this case, a series of log-returns, one per timestamp.

Only after this projection step the generator can assemble the final output that is passed to the discriminator: the timestamps are paired with their corresponding values $\{y_{t_i}\}$ creating a path $Y_t = (t, y_t)$, and a piece-wise linear interpolation is constructed. This continuous-path representation ensures that the discriminator's Neural CDE can ingest the generated series as a smooth signal rather than discrete samples.

Gradient Computation with Adjoint Sensitivity (Backpropagation)

When using the Euler-Maruyama solver, unlike some symmetric solvers, the updates are generally not algebraically reversible, meaning past states X_i cannot be exactly reconstructed from future ones without additional computation or storage.

To enable gradient-based training, once the forward pass reaches the final time during training, gradients of the loss are computed by integrating an adjoint SDE backward in time from t_N to t_0 along the same temporal grid.

This adjoint SDE captures the sensitivity of the loss with respect to the parameters (θ) of the neural networks defining the drift and diffusion. At the end of this backward pass, the adjoint state at t_0 yields the exact gradients $\partial L/\partial\theta$, which are then used to update the generator's neural network parameters.

Crucially, this adjoint method offers a significant advantage in memory efficiency because it avoids the need to store the entire forward trajectory. However, the noise path (the sequence of Brownian increments) from the forward pass often needs to be stored or deterministically reconstructed for the backward pass to ensure consistency.

This method is implemented in libraries such as *torchsde* via the *sdeint_adjoint* interface, making it a practical and scalable choice for training Neural SDEs in complex generative settings.

Discriminator (Critic)

After detailing the generator's stochastic integration mechanism, the next step is to define the discriminator's counterpart, an architecture that treats each generated or real path as a continuous signal and learns to distinguish them via a Neural Controlled Differential Equation.

In this discriminator, the continuous path Y_t reconstructed from the interpolation coefficients acts as the "control" driving the hidden state h_t.

At the first timestamp t_0 the discriminator evaluates the observed value Y_{t_0}, which includes both time and data, through a small MLP to produce an initial hidden state $h_{t_0} = h_0$. This ensures the very start of the trajectory is encoded in the latent state.

For $t > t_0$, the hidden state evolves according to the Controlled Differential Equation (CDE):

$$h_t = h_0 + \int_{t_0}^{t} f(s, h_s)\, dY_s$$

where $f(s, h_s)$ is the learned vector field that, at each moment, reads in the current time s, current hidden state h_s, and maps the infinitesimal change dY_s of the input signal into an update for h. Here dY_s captures both the increment in the actual data value and the passage of time. In other words, rather than just looking at each data point in isolation, the model continuously updates its internal memory in response to every infinitesimal movement of the input.

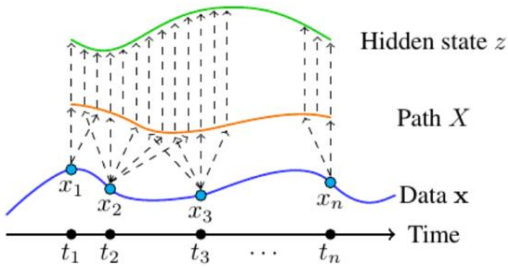


Figure 7: Neural CDE dynamics, hidden state evolution from input path
Source: see Kidger, P et al (2020) on Neural CDEs

The blue curve represents the discrete observed inputs $y_1, y_2, \ldots, y_n$. The orange line denotes the reconstructed continuous path Y_t, obtained via interpolation. The green curve illustrates the evolving hidden state h_t, which is continuously updated by the CDE in response to changes in the input path.

The integration step is what actually "runs" the neural CDE, it is the mechanism by which the discriminator reads the continuous path and transforms it into a representation suitable for classification.

Rather than computing that integral analytically, the discriminator uses a fourth-order Runge–Kutta scheme (RK4) over each interval.
As soon as the generator hands off a fully specified path Y_t defined by its interpolation coefficients, the discriminator's evolution becomes purely deterministic. Therefore because Y_t is no longer random, the integral is an ordinary controlled ODE, and RK4 is both valid and efficient for its numerical solution.
A fixed time step of Δt = 0.5 is used as this higher order method does not require as small time steps as Euler–Maruyama to achieve the same level of accuracy.

At each interval time, the solver evaluates the learned vector field f at four carefully chosen points, namely the interval endpoints and two mid-points, and then combines those slope estimates in a weighted average to step the hidden state forward with fourth-order accuracy. This process is repeated across the entire time grid, producing a sequence of latent states $\{h_{t_0}, h_{t_1}, \ldots, h_{t_N}\}$ that faithfully track even small variations in the input path.
Once integration reaches the final timestamp t_N, the discriminator applies a linear readout to a summary of the hidden trajectory, which includes both the mean and standard deviation of h_t over time, collapsing it to a scalar "real vs. fake" score, through a simple linear layer.

By treating the generated or real series as a control signal and solving a neural CDE against it, the discriminator naturally absorbs information at every infinitesimal increment of time and value, making it particularly powerful for distinguishing subtle differences in continuous-time trajectories.

3.3 Training Procedure

With both generator and discriminator architectures defined, the next step is to understand how they are trained jointly. Here, the generator and discriminator engage in an adversarial game: the discriminator learns to assign higher scores to real trajectories and lower scores to generated ones using a Wasserstein-1 loss with a gradient penalty, while the generator learns to maximize its critic score.

Gradient Penalty

The use of Lipswish activations already bounds each layer's slope, imparting Lipschitz control on the network. This was complemented with the classic gradient penalty which softly constrains the norm of the critic's input–gradients toward unity along interpolations between real and fake samples. In practice, this combination proved superior to alternatives like spectral normalization or aggressive weight clipping, which were either too weak to prevent exploding gradients or so restrictive that learning stalled. By pairing the bounded slope of Lipswish with the targeted regularization of the gradient penalty, the critic maintains a well-conditioned, stable Lipschitz constant throughout training, yielding smoother dynamics and higher-quality samples.

For each real trajectory x and its generated counterpart $\tilde{x}$ the method samples $\varepsilon \sim U(0,1)$ and forms the interpolation, $\hat{x} = \varepsilon\, x + (1 - \varepsilon)\tilde{x}$.
It then computes the gradient of the critic's score with respect to $\hat{x}$, $\nabla_{\hat{x}} D(\hat{x})$, evaluates its Euclidean norm $\| \nabla_{\hat{x}} D(\hat{x}) \|_2$, and adds the term:

$$GP = \lambda\, \mathbb{E}_{\hat{x}}(\|\nabla_{\hat{x}}\, D(\hat{x})\|_2 - 1)^2$$

to discriminator loss. This regularization ensures $\| \nabla_{\hat{x}} D(\hat{x}) \|_2 \approx 1$.

Training Loop

Before any training begins, the full set of time-series is split into a training portion (80 %) and a held-out test set (20 %) so that final performance reflects true generalization. A PyTorch *DataLoader* is then constructed to iterate over batches of fixed-size windowed sequences. Setting Shuffle=True randomizes the order of the windowed sequences at the start of each epoch,

before forming batches. This is appropriate because, although the temporal structure within each sequence must be preserved, there's no need to keep a fixed order between sequences. It helps prevent the model from overfitting to the presentation order and encourages better generalization.

During each epoch, the training loop cycles through batches of windowed log-return sequences together with their shared timestamp grid. For every batch, the real data are first converted into continuous-path coefficients for the discriminator via piecewise-linear interpolation. The discriminator then receives two updates for each generator update. Because the generator's update relies entirely on the critic's feedback, it's crucial that the critic remains a reliable evaluator of the current fake distribution. Giving the critic extra update steps lets it "catch up" whenever the generator changes its output, preventing the generator from chasing a moving, under-trained target. This balance yields steadier loss curves and avoids collapse.
With the generator frozen, the critic is trained with the Wasserstein-1 loss plus a gradient penalty and it scores both real and freshly generated paths.

$$\mathcal{L}_D = \mathbb{E}_{\tilde{x}\sim p_G}[D(\tilde{x})] - \mathbb{E}_{x\sim p_{real}}[D(x)] + GP$$

In practice, it is estimated by calculating the difference between its average score on fake samples and its average score on real samples. The gradient loss is then back-propagated and the critic's parameters are updated via Adam optimiser.

Once the critic has been updated, the generator produces a fresh batch of trajectories by integrating its learned SDE from random noise. Those synthetic paths are then interpolated and scored by the critic. The generator's objective is to make the critic assign high scores to fake paths, which is implemented by minimizing:

$$\mathcal{L}_G = -\mathbb{E}_{\tilde{x}\sim p_G}[D(\tilde{x})]$$

Gradients of $\mathcal{L}_G$ flow through the adjoint-SDE solver, updating the MLPs that parameterize the SDE's drift, diffusion, and readout layers.

From epoch 70 onward, stochastic weight averaging is applied to both networks using *torch.optim.swa_utils.AveragedModel* from PyTorch library and it allows to maintain a running average of the model weights across iterations. This technique encourages more stable convergence and helps improve generalization and consistency of the learned dynamics.

That ordering in the training setup is intentional and common practice in GAN training: you update the discriminator one or more times first so that it remains a strong, informative critic before you update the generator.

Training Dynamics and Monitoring

Throughout the training process and each experimental run, careful monitoring was essential to navigate the inherent instabilities often encountered in adversarial learning frameworks, particularly with complex generative models like Neural SDEs.

At each epoch, and frequently at finer intervals, several key indicators were logged and reviewed. These included the generator and discriminator loss values (specifically, the critic scores for real and fake samples and the overall WGAN-GP loss), the L2 norms of the gradients for both networks ($\| \nabla D \|, \| \nabla G \|$), and the mean absolute deviation of the discriminator's gradient norm from 1 ($Mean \mid \| \nabla D/\partial x \| - 1|$) to assess the effectiveness of the gradient penalty. Other diagnostic prints also provided insights into the SDE dynamics, such as the standard deviation of the generated versus real log-return increments (Δy std), and the evolution of the learnable scales *scale_f* and *scale_g*. Periodically, generated samples were also qualitatively inspected to correlate quantitative metrics with perceptual realism.

Encouragingly, throughout most training runs, the gradient norms for both the generator and discriminator remained balanced and did not exhibit signs of explosion, suggesting that the gradient penalty and discriminator gradient clipping were effective in maintaining a degree of stability in the updates.

Despite this underlying stability in gradient behavior, a common pattern observed across many training runs was a cyclical or non-monotonic improvement in generated sample quality. For instance, the model might appear to generate plausible trajectories around certain epochs, only for the quality to degrade in subsequent epochs, before potentially recovering or collapsing into less diverse outputs. These oscillations often signaled a temporary imbalance between the generator and discriminator, where one network might gain a temporary advantage, leading the other to overcorrect its gradients and become less informative.

Such observations were critical in guiding iterative adjustments to hyperparameters, including the relative learning rates of the generator and discriminator, the D:G update ratio, and the initialization or constraints on the scales for drift and diffusion, in an effort to achieve a more stable training equilibrium, non-exploding gradients and consistently high-quality generative performance.

3.4 Hyperparameters Tuning

Because GANs are notoriously sensitive to their configuration, careful hyperparameter tuning becomes the single most critical step in driving the success of the entire simulation. Small adjustments to learning rates, network widths, or penalty weights can make a huge difference

between stable convergence and collapse, and thus systematic exploration and validation of these settings are essential to achieve reliable synthetic trajectories.

Parameter	**Value**
Sequence length (T)	50
Batch size	64
Data dimension	1
Initial noise dimension	16
SDE noise dimension	4
Hidden-state size	32
MLP width	64
Number of hidden layers	2
Generator learning rate (Adam)	0.0002
Discriminator learning rate (Adam)	0.0002
Gradient-penalty weight (λ)	10
Epochs	100
SWA start epoch	70

Table 1: Hyperparameters

Sequence length was set to 50, this parameter defines the number of discrete points in each sequence that the model aims to generate and consequently the length of the generated time series. This choice was made as a balance between capturing sufficient temporal structure and maintaining computational efficiency. Financial time series such as daily log-returns often exhibit short- to medium-range dependencies, meaning that local dynamics over periods of around two months (≈50 trading days) can be highly informative for learning patterns.

A batch size of 64 was used during training. This is a common value in deep learning that balances stable gradient estimation for the stochastic optimization process with computational efficiency and memory constraints.

The model was configured with a data dimension of 1, designating its application to generating univariate time series. This means that while the model outputs a sequence of observations over time, such as our simulated stock price path, the value of the observation y_t at each individual time point in the trajectory is a single scalar.

The internal architecture of the Neural SDE and its associated neural network components were defined by several key dimensionalities and structural choices, refined through preliminary experimentation. An initial noise dimension of 16 was used for the latent vector seeding the

generator, providing sufficient capacity to map to diverse initial SDE states (X_0) and promote varied sequence generation. For the SDE dynamics, an SDE noise dimension of 4 was selected. This choice aligns with findings in financial modeling where multi-dimensional Brownian motions are standard for capturing complex market dynamics. Preliminary investigations also indicated that a scalar Brownian motion was insufficient to replicate key stylistic features of the target data, while dimensions beyond 4 did not yield commensurate improvements to justify the increased complexity of the diffusion term. The core hidden-state size (X_t) of the Neural SDE was set to 32. This dimensionality still consistent with common practices was validated in our setup where smaller dimensions (e.g., 16) struggled to capture, whereas larger dimensions (e.g., 64) increased training times without a significant enhancement in key performance metrics. Finally, the Multi-Layer Perceptrons (MLPs) parameterizing the drift and diffusion functions, as well as the other auxiliary networks, utilized 2 hidden layers, each with a MLP width of 64 units. This network depth and width were found to provide robust approximation capabilities for the SDE's governing functions.

The optimization of both the generator and discriminator networks was performed using the *Adam optimizer*. Adam is an adaptive learning rate optimization algorithm that adjusts the learning rate for each parameter in the model individually. It does this by looking at past changes, like tracking momentum and how much gradients have varied, which is really helpful for tricky models like GANs that have constantly shifting learning goals.
Initial experiments with other adaptive methods, such as Adadelta, did not yield stable training dynamics for this particular architecture. Despite Adadelta's design for automatic learning rate adjustment, issues related to training stability, such as oscillating losses or divergence (exploding trajectories), were encountered. Adam, configured with a learning rate of 2e-4 for both networks, a *beta1* parameter of 0.5 to make momentum more responsive to recent gradients, and *beta2* at its common default of 0.999, offered a more robust and reliable training process.

A weight (λ) of 10 for Gradient Penalty was chosen following standard practice in literature where it has been shown to provide good training stability, and it proved effective.

The model was trained for a total of 100 epochs, which provided sufficient training exposure, and Stochastic Weight Averaging (SWA) was initiated at epoch 70. Activating SWA in the later stages of training allows the model to average weights in a region of the loss landscape to which the optimizer has likely already converged, often leading to solutions with better generalization performance.

Chapter 4: Empirical Evaluation

This chapter presents the empirical results obtained from training and evaluating the GAN-based Neural SDE model on historical financial data. The primary objective is to assess the model's ability to generate realistic log-return trajectories that replicate the statistical properties of the S&P 500 index. Following the methodological framework described in the previous chapter, the model is evaluated using a combination of qualitative analyses and quantitative metrics. The results highlight the model's strengths in capturing typical market behavior, as well as its ability in generalizing to regimes not seen before.

4.1 Evaluation Metrics

In order to evaluate the performance of the GAN-based Neural SDE model developed, a set of distributional and temporal metrics that are particularly relevant for financial time series analysis were selected. Given the importance of generalization in generative models, these metrics have been applied separately to the training and test sets to better understand the model's ability to replicate realistic financial behavior without overfitting to data used during training phase.

Log-returns distribution

The marginal distribution of log returns is a fundamental aspect of financial time series. For both real and generated data, KDE plots were computed to visually compare their shapes. Heatmaps of the distribution across time were also generated to visually assess how well the generator preserves the temporal structure and dispersion of the returns.

Additionally, first-order statistics were extracted, including:

- *Mean* and *standard deviation* to capture location and scale.
- *Skewness* to detect asymmetry.
- *Kurtosis* to assess the heaviness of tails.

Autocorrelation Function (ACF)

The autocorrelation function of log returns was computed up to a specified lag to evaluate the degree of memory in the time series. Since log returns are typically assumed to be weakly autocorrelated in efficient markets, the generator is expected to replicate a similar structure. Deviations in the ACF of the generated data may indicate temporal artifacts or overfitting to specific patterns in the training set.

As a quantitative assessment, the mean squared error (MSE) between the mean ACFs of real and generated sequences was computed. This metric provides a compact summary of how well the temporal dependence structure is preserved by the model.

Qualitative trajectory comparison

A visual inspection of sample paths was conducted to assess the realism and diversity of the simulated trajectories. This includes direct overlay plots of generated vs. real paths, as well as visual comparisons of volatility regimes and local trends. Although qualitative, this evaluation provides intuitive insight into the model's ability to generate realistic scenarios.

Maximum Mean Discrepancy

The MMD is a kernel-based statistical distance used to compare two distributions without requiring explicit density estimation. In this context, MMD is computed between batches of real and generated trajectories using a Gaussian kernel, treating each trajectory as a single vector in $\mathbb{R}^T$. It measures how similar the distributions of the entire sequences are across time. This metric is widely used in GAN-based evaluations. A lower MMD value indicates higher similarity.

KS and Wasserstein Distance

The Kolmogorov-Smirnov (KS) test assesses how similar two distributions are by finding the maximum point of divergence between their cumulative distribution functions (CDFs). It's particularly sensitive to shifts in location and shape, especially around the median. A small KS statistic (and a high p-value) indicates good similarity.

The Wasserstein Distance (WD), often called "earth mover's distance," instead, measures the minimum effort needed to transform one distribution into the other. It's valuable because it considers the entire distributions and is particularly good at reflecting differences in their overall shape, including variations in spread or tail behavior.

Together, these metrics form a comprehensive suite that supports both statistical validation and practical applicability in financial modeling tasks.

4.2 Evaluation Setup and Sample Extraction

As during training setup, to evaluate the model's ability to reproduce realistic financial dynamics, both the training and test datasets are processed using a sliding window approach. This transforms the raw time series of daily log-returns into fixed-length sequences of L = 50 time-steps, spanning from t = 0 to t = 49, where t denotes the relative index within each trajectory. For the evaluation on training set, 1000 non-overlapping real trajectories are extracted from the

corresponding period while 1000 synthetic ones are generated for comparison. Similarly, for the test set evaluation, 300 synthetic trajectories are generated from the trained Neural SDE model and directly compared to 300 real sequences.

To ensure a valid one-to-one comparison of distributional and temporal characteristics, unlike the training phase, no shuffling is applied during evaluation phase. This preserves the temporal order and consistency of real samples and aligns the structure of generated and empirical sequences. The resulting samples, identical in shape and duration, allow for meaningful metric-based comparisons across both datasets.

Additionally, log-returns are left in their scaled form during evaluation. Since all metrics are computed jointly on real and generated data, and the evaluation focuses on the alignment of distributional properties, scaling does not compromise the validity of the comparisons.

4.3 Evaluation on Train Data

- *Visual Distribution Comparison*

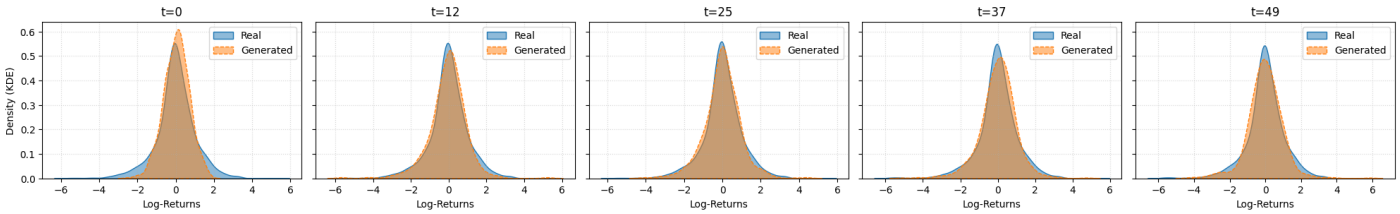


Figure 8: Marginal Distributions of Real vs Generated Log-Returns

Figure 8 shows a sequence of Kernel Density Estimate (KDE) plots comparing the scaled log-return distributions of real and generated data at five selected time steps (t = 0, 12, 25, 37, 49). The generated distributions (orange) closely match the real ones (blue) across all selected time steps. The real distributions show slightly heavier tails, but the overall discrepancy is minimal. Both sets exhibit a sharp peak around 0 and an approximately symmetric bell shape.

There's no visible mode shift, distortion, very heavy tails or multimodal behavior in the generated distributions this suggests that the model has successfully learned the shape of the log-returns observed during training.

This first visual inspection suggests that generated marginals are well-calibrated, preserving the mean, variance, symmetry, and kurtosis characteristics of the real data.

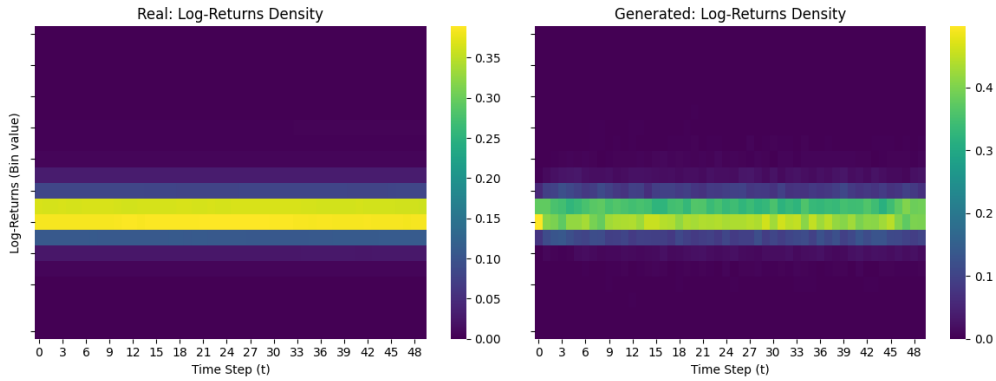


Figure 9: Heatmap of Real vs Generated Log-Returns Distributions

To further support this analysis, the heatmap of log-returns densities (*Figure 9*) offers a comprehensive time-resolved view. The generated data captures the overall shape well but shows slightly higher variability and broader dispersion across bins. This suggests that the generative model is able to approximate the temporal marginal structure, though with minor deviations in tail behavior and overall spread.

- *Visual Comparison of Trajectories*

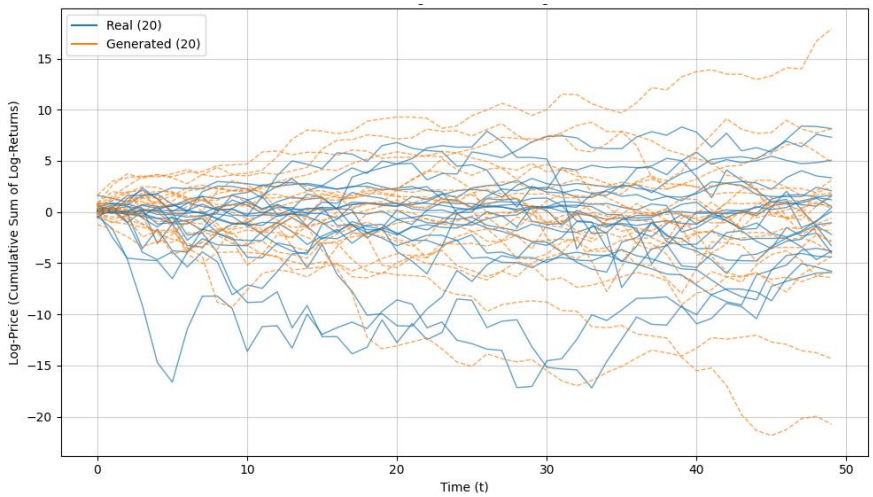


Figure 10: Log-Price Trajectories, Cumulative Sum of Log-Returns Over Time

Real trajectories (blue) in *Figure 10* display a realistic variety, some trending upwards, others downwards, with moderate volatility. Generated trajectories (orange, dashed) span a similar range and look qualitatively consistent, but, there is slightly more spread in the generated trajectories, particularly towards the end of the time horizon. Some generated paths appear a bit more erratic or volatile, suggesting higher variability and uncertainty in the learned dynamics.

Importantly, there's no collapse to zero or explosive behavior. Generated paths remain diverse and active, with realistic cumulative dynamics.

It is crucial for such a model to demonstrate that it can generate diverse trajectories without collapsing into a single dominant mode or drifting uniformly. A generative model that captures only the average behavior would fail to reflect key properties of financial markets, such as volatility clustering, trend uncertainty, and regime shifts.

- *Autocorrelation Structure Comparison*

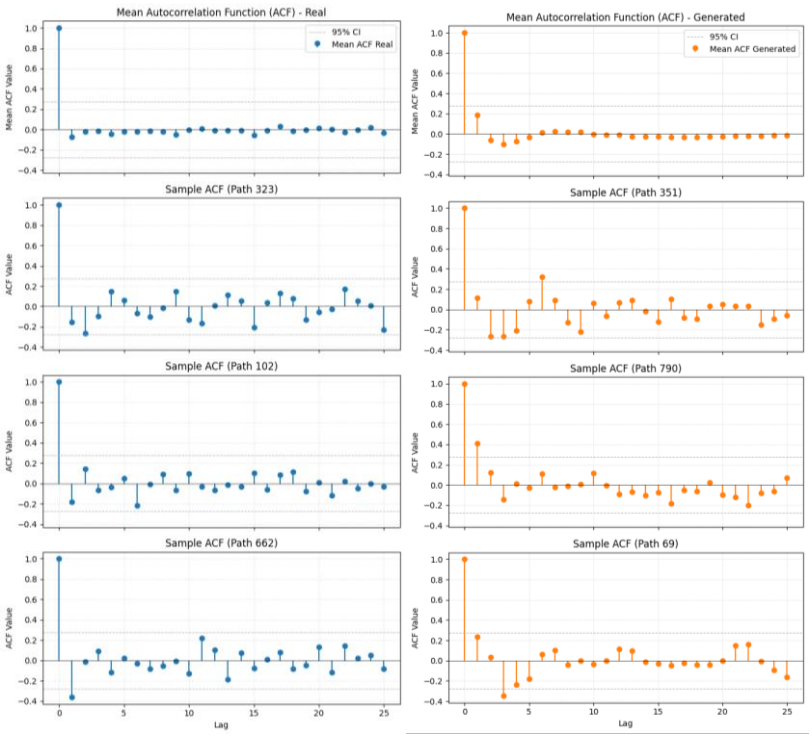


Figure 11: Autocorrelation Function (ACF) plots of real and generated log-returns.

ACF plots in *Figure 11* compare the autocorrelation structure of real and generated log-returns. As expected for financial time series, real log-returns exhibit negligible autocorrelation beyond lag 0, with mean ACF values oscillating around zero and most sample paths remaining within the 95% confidence interval.

The generated data shows a similar behavior, mean ACFs are near zero for all lags beyond the first, with no significant autocorrelation patterns. The model successfully learned to reproduce the near white-noise temporal structure typical of real log-returns, avoiding artificial dependencies across time.

Metric	Value
MSE of Mean ACFs (Lags 1-N)	0.0040

Table 2: MSE of Mean ACFs

A Mean Squared Error (MSE) close to zero indicates that, on average, the autocorrelation structures of the generated series are very similar to those of the real series across the specified lags. This very small value suggests that the generator is doing an excellent job of capturing the linear temporal dependencies present in the real financial time series, and it demonstrates that the model is not just matching marginal distributions but is also replicating important aspects of the time-dependent structure.

- *Quantitative Metrics*

	Mean_Real	Mean_Gen	Std_Real	Std_Gen	Skew_Real	Skew_Gen	Kurt_Real	Kurt_Gen
Time (t)								
0.0	-0.016	0.048	1.047	0.669	-0.413	-0.201	3.103	0.157
12.0	-0.013	-0.047	1.046	0.961	-0.419	-0.540	3.134	4.845
25.0	-0.005	-0.008	1.034	0.936	-0.406	-0.134	3.268	2.349
37.0	-0.016	0.019	1.074	0.952	-0.612	-0.469	3.919	2.925
49.0	-0.014	-0.021	1.080	0.956	-0.605	-0.130	3.795	3.444

Table 3: Comparison of marginal moments (mean, standard deviation, skewness, kurtosis) between real and generated log-returns at selected time steps.

Table 3 reports the evolution of the main four marginal moments, mean, standard deviation, skewness, and kurtosis, for both real and generated log-returns at five key time steps.
Both series are centered around zero, as expected. While the signs occasionally differ in generated means, the differences are very small and do not suggest systematic bias over time.
In terms of standard deviation, the generator initially underestimates the volatility, but gradually aligns with the real values at later steps, reflecting improved calibration over time.
The model also captures the negative skewness, reflecting the left-tailed nature of financial return distributions, maintaining the correct sign across all time steps. Although the magnitude is not always exact, the overall asymmetry is preserved.
Regarding kurtosis, the model generally reproduces the heavy tails observed in real data, though with some irregularities. At $t = 0$, kurtosis is clearly underestimated (0.157 vs. 3.103), whereas at $t = 12$, it is slightly overestimated (4.845 vs. 3.134), suggesting that the representation of extreme values may vary across time.

Overall, the results indicate that the model captures the key distributional features of financial log-returns, with more accurate moment matching emerging in the later stages of the sequence.

	KS_stat	KS_p	WD
0.0	0.068	0.0196	0.230
12.0	0.044	0.2878	0.089
25.0	0.030	0.7594	0.075
37.0	0.069	0.0171	0.106
49.0	0.045	0.2635	0.104
Mean over t	0.051	0.2695	0.121

Metric	Value
MMD^2 (RBF σ=1.0) Trajectories	0.002000

Table 4: Goodness-of-fit metrics comparing real and generated log-returns.

The KS statistic values in *Table 4* remain low (mean ≈ 0.051), suggesting that the distributions are closely aligned. Moreover, the associated p-values, most of which are above the common 0.05 threshold, indicate that in several cases we cannot reject the null hypothesis that the real and generated samples come from the same distribution. Although a few p-values fall below 0.05, their corresponding KS statistics remain small, implying minor

deviations. The Wasserstein distances also stay within a very low range (mean ≈ 0.12), reinforcing this conclusion.

At the trajectory level, the squared Maximum Mean Discrepancy (MMD^2) with an RBF kernel returns a very small value (0.0020), suggesting that the overall distribution of entire generated paths is highly consistent with the real ones.

4.4 Evaluation on Test Data

- *Visual Distribution Comparison*

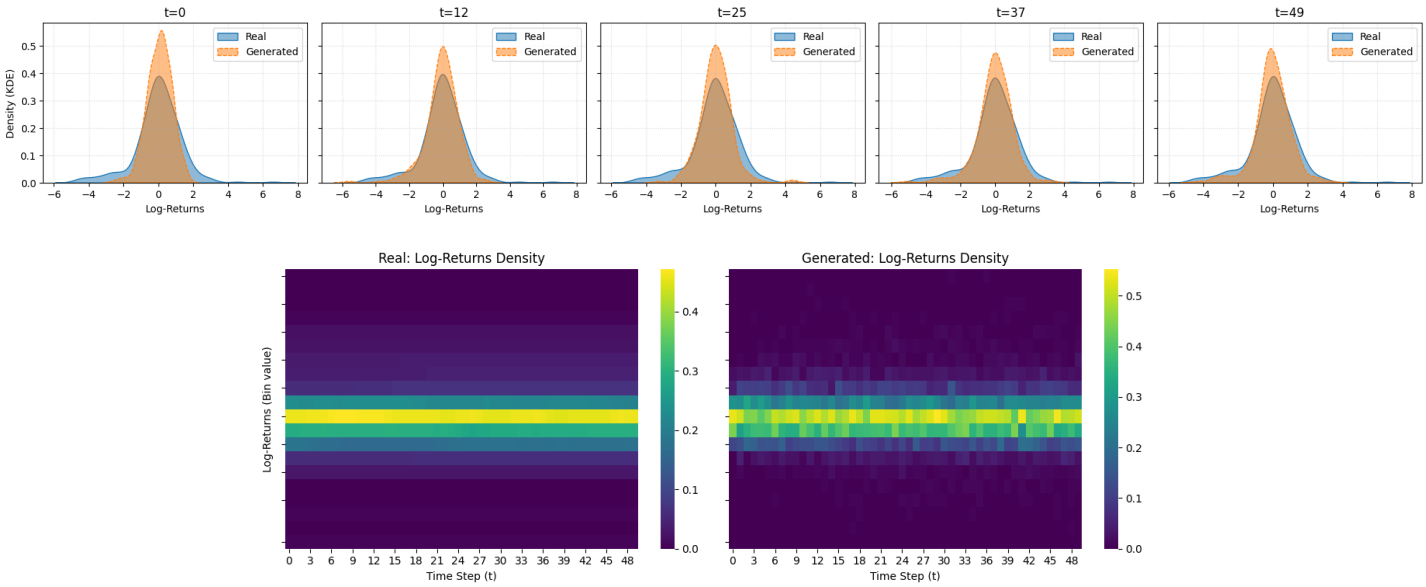


Figure 12: Distribution comparison on test data.
Top: Histograms of real vs. generated log-returns marginal distributions at selected time steps.
Bottom: Heatmaps comparing the temporal evolution of the distributions for real (left) and generated (right) samples.

The marginal distribution plots (*Figure 12*) on the test set indicate that the model generalizes fairly well beyond the training data, successfully capturing key statistical features of real log-returns with reasonable accuracy. Across all selected time steps, the generated distributions reproduce the overall bell-shaped structure and central peak around zero, with progressively better alignment as time evolves.

The generated distributions show a slightly taller central peak than the real ones, indicating that the model slightly concentrates probability mass around the mean, sometimes at the expense of the tails.

One particularly important aspect is that real data consistently exhibit negative skewness, reflecting the presence of more extreme negative returns. The generated distributions preserve this skewed shape, especially at $t = 37$ and $t = 49$, where the left tail is visibly heavier than the right, closely mirroring the real data. At earlier time steps (e.g., $t = 0$), the model slightly underestimates dispersion and consequently tail mass, but without introducing distortions or mode shifts.

The heatmap created for test data (*Figure 12*) shows a clear yellow band around the central region of the generated log-return distribution, indicating good alignment with the mode of the real data. The color bands are a bit more dispersed vertically and less smooth than real ones. Compared to training, the structure remains coherent, and this denser concentration toward the mean suggests the model is effectively capturing the most probable behaviors, even when generalizing beyond the training set. The Neural SDE has learned a stable stochastic evolution.

- *Visual Comparison* of *Trajectories*

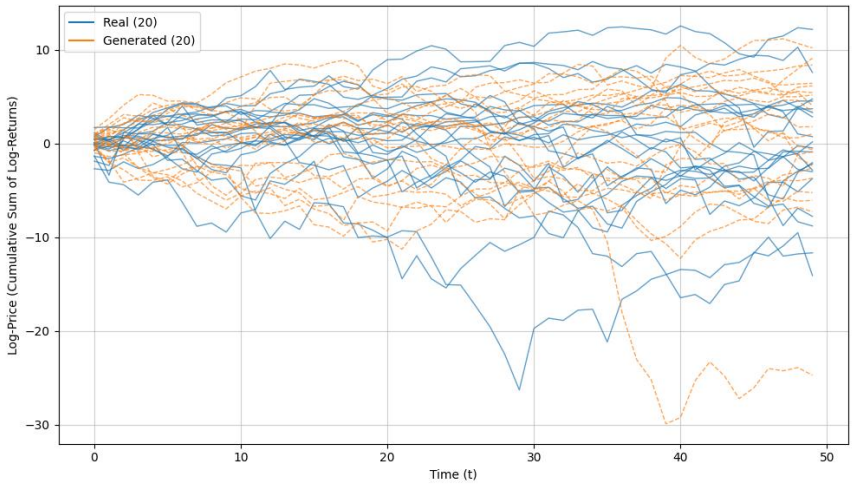


Figure 13: Test set Log-Price Trajectories, Cumulative Sum of Log-Returns Over Time

On the held-out test set, the real (blue) and generated (dashed orange) cumulative log-return paths exhibit a very similar, but not perfectly matched, overall dispersion and range (*Figure 13*). Both sets include trajectories that drift up, drift down, and remain roughly flat, with comparable magnitudes of extreme moves. The overall shape is well preserved. The model clearly hasn't collapsed to a single "average" path, instead it faithfully produces a spectrum of behaviors, which is very important for this kind of model. However, the very most extreme negative swings at mid-horizon are slightly under-represented. Overall, the generated trajectories on test data capture the volatility and trend uncertainty, it may still fall a bit short in replicating the full extent of tail risk present in the real series.

- *Autocorrelation Structure Comparison*

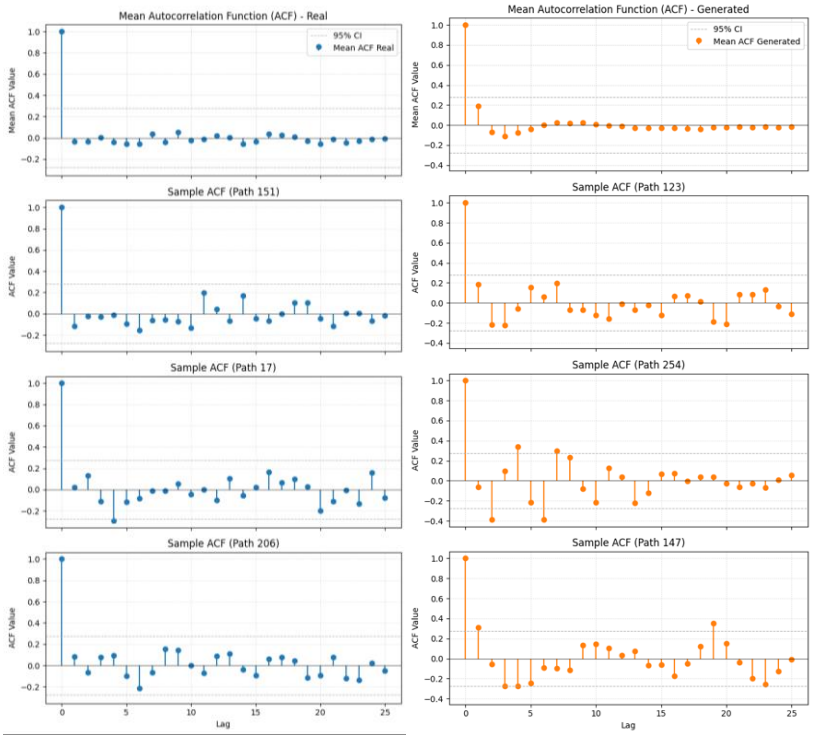


Figure 14: Autocorrelation Function (ACF) plots of real and generated log-returns on Test Data

Both the real and generated series show the classic "white-noise" decay in autocorrelation (*Figure 14*): a spike at lag 0 and then rapid fall off toward zero. The synthetic log-returns exhibit higher ACF at first lags compared to the real data, suggesting the model retains a bit more short-term memory than the market actually does. Beyond the first few lags, however, both mean ACF curves hover around zero and lie within the 95 % confidence bounds.

Metric	Value
MSE of Mean ACFs (Lags 1-N)	0.0036

Table 5: MSE of Mean ACFs on Test Data

Overall, the very low MSE between the mean ACFs confirms that the generator has learned the market's autocorrelation structure with high fidelity on unseen data, demonstrating a genuinely learned temporal behavior rather than mere memorization of training noise.

- *Quantitative Metrics*

Time (t)	Mean_Real	Mean_Gen	Std_Real	Std_Gen	Skew_Real	Skew_Gen	Kurt_Real	Kurt_Gen
0.0	-0.026	0.066	1.344	0.702	-0.332	-0.377	3.422	0.387
12.0	-0.042	-0.047	1.339	0.966	-0.307	-1.094	3.484	4.160
25.0	-0.045	0.012	1.365	0.887	-0.305	0.589	3.094	3.888
37.0	-0.041	-0.020	1.363	1.001	-0.314	-0.841	3.135	3.437
49.0	-0.031	-0.030	1.363	1.011	-0.334	-0.609	3.149	2.579

Table 6: Comparison of marginal moments on Test Data (mean, standard deviation, skewness, kurtosis) between real and generated log-returns at selected time steps.

Turning now to the marginal moments on the test set (*Table 6*), the synthetic data closely mirrors the overall real log-return characteristics over time. The average returns cluster around zero just as in the real data. Generated standard deviations (0.70–1.01) are lower than the real ones

(≈1.34–1.36), meaning the model produces slightly smoother paths on unseen data. It still captures the general volatility regime but tends to understate the full range of fluctuations.
It is worth noting that the test set itself exhibits higher realized volatility (σ ≈ 1.35) than the training set (σ ≈ 1.05), presenting a regime the model had not encountered during training.
Where we see the biggest ripples is in the higher moments. The market's real returns are mildly left-skewed across time, but the generator swings between over and under skewing, especially at t=12 where it overshoots into strong negative skew and at t=25 when it flips positive. The same goes for kurtosis: real returns sit comfortably in the 3–3.5 range, yet the model starts off too flat at t=0 before overestimating it in the mid horizon (hitting above 4). In summary, the model is doing a solid job capturing average behavior, but it occasionally misjudges when predicting how heavy or light the extremes should be.

	KS_stat	KS_p	WD
0.0	0.117	0.0336	0.383
12.0	0.077	0.3417	0.235
25.0	0.113	0.0423	0.304
37.0	0.083	0.2488	0.236
49.0	0.087	0.2101	0.239
Mean over t	0.095	0.1753	0.279

Metric	Value
MMD^2 (RBF σ=1.0) Trajectories	0.006667

Table 7: Goodness of fit metrics comparing real and generated log-returns on Test Data

Most of the KS p-values (*Table 7*) exceed the usual 5% threshold, meaning we can't reject the hypothesis that the real and generated marginals come from the same distribution at those times-steps. The slightly elevated KS stats and WD compared to training hint at minor shifts, but nothing dramatic.

Finally, the trajectory-level MMD^2 rises to ≈0.0067 on test versus ≈0.0020 on train, suggesting that while the generator still captures the overall path structure, it shows a bit more deviation when facing unseen sequences.

Altogether, these results demonstrate that the Neural SDE model developed is able to generalize well on unseen data, capturing both marginal and temporal dynamics of real log-returns without overfitting. It can be considered a credible tool for realistic synthetic data generation, although with minor adjustments.

The synthesis of these achievements confirms that the Neural SDE-GAN built is capable of enhancing traditional methods of financial modeling, portfolio management, and risk evaluation through the provision of high-quality and statistically accurate synthetic scenarios.
While these quantitative and visual results are promising, there are still open questions. The next chapter will delve into the model's limitations, such as tail risk underestimation and sensitivity to market regime changes and propose future work on improved drift/diffusion estimation, enhanced volatility clustering, and multi-factor extensions.

Chapter 5: Discussion and Future Research Directions

Developing the generative model with the adopted framework inherently presented a series of significant challenges. As anticipated, with an architecture of this complexity, successful implementation required careful hyperparameter tuning, meticulous attention to several interacting components, and an iterative refinement process to problem-solving. This section critically analyzes the key obstacles encountered, the effectiveness of the strategies employed to overcome them, and identifies its current limitations and avenues for further improvement.

5.1 Navigating Computational Complexity and Training Instability

A primary operational obstacle throughout this iterative process was the substantial training time required for each experimental run. GANs, by their nature, are computationally intensive. Each adjustment, whether to model architecture, SDE parameters, or optimizer settings, necessitated every time a retraining cycle to accurately measure its impact.

This significant training duration for the SDE-GAN is primarily due to its core mechanics. For instance, the generator doesn't simply perform a one-shot calculation; for every sample window in a batch, it must numerically simulate the financial time series step-by-step using the solver over the entire sequence length, and, calculating the gradients involves an equally intricate backward pass through this entire simulation. Similarly, the discriminator also processes the entire trajectories using its own solver. These intensive computations for both networks, repeated for every sample across many training epochs, naturally accumulate. This is compounded by the nature of adversarial learning itself: the generator and discriminator often require an extended period of co-evolution to stably learn the statistical properties and temporal dynamics, hence, before one can reliably assess whether a given model configuration is truly effective or showing genuine promise, a substantial portion of the total planned training duration must be completed. As a result, the experimental cycle demands patience, careful planning of experiments and efficient use of available computational resources.

The choice of the time step Δt for the Euler–Maruyama solver, for instance, was also driven by computational considerations. While smaller step sizes could have improved numerical convergence and simulation accuracy, they would have significantly increased training time. A balance was therefore necessary to ensure reliable approximation of the underlying dynamics while keeping the computational burden manageable.

Initial attempts at training the SDE-GAN began with modeling the dynamics of log-prices directly. However, this approach introduced notable instability. Log-prices are inherently non-

stationary, which complicates learning and increases the risk of mode collapse. In early experiments, the model frequently failed to capture the multi-modal structure observed in real log-price distributions, instead producing oversimplified, unimodal trajectories that poorly reflected the diversity of actual market behavior.

Subsequently, even after shifting focus to the more stationary log-returns, a key challenge persisted: a pervasive form of mode collapse in which the generator consistently produced flat or low-variance trajectories. Addressing this required extensive tuning of training dynamics, such as learning rates and the update ratio between discriminator and generator. If either network advances too quickly, especially the discriminator, training can stall entirely, with one side failing to learn meaningful gradients.

Getting the starting points (x_0) for the SDE paths right was critical. It was observed that if these initial states didn't have enough variety, the generator would get stuck producing very similar, uninteresting financial series, a classic case of mode collapse. In particular, the model tended to map all noise realizations to a nearly identical initial point, often close to zero, causing all generated paths to start from the same location. Hence, a lot of effort went into making sure the initial network (the one mapping noise to x_0) consistently gave the SDE a rich set of diverse starting conditions. Even then, the need to keep an eye on it persisted, as the model sometimes tries to 'unlearn' this randomness during training.

Furthermore, the selection of an appropriate optimization algorithm itself proved to be a significant factor. While adaptive methods like Adadelta were initially considered, empirical results showed it struggled to stabilize this specific SDE-GAN architecture, leading to persistent training issues such as unreliable convergence and a tendency for generated trajectories to exhibit apparent stability before suddenly collapsing or exploding. A shift to the Adam optimizer was necessary. This underscore the importance of optimizer selection and highlight the need for more tailored training strategies in adversarial training frameworks.

5.2 Future Improvements and Directions

Building upon the current model's achievements and the insights gained during its development, several promising directions for future work could further improve the accuracy, realism, and utility of the generated financial time series. The current model's limitations naturally guide these prospective enhancements.

One key area for refinement involves improving the statistical fidelity of the generated series at a granular level. The observed discrepancies in the initial log-return (t=0) and the slight deviations in skewness and kurtosis across time suggest opportunities for more targeted learning. Future work could introduce explicit moment-matching terms into the generator's loss function, providing direct signals to correct deviations in these specific statistics. Complementing this, investigating specialized mappings, such as deeper MLP or normalizing flows, or conditioning mechanisms for the initial SDE state (x_0) could specifically address the t=0 mismatch, potentially allowing the SDE to start its path generation from a more accurately calibrated starting point.

Further advancements could focus on achieving better control over the global properties of generated trajectories, particularly cumulative path behavior and long-term drift. While one-step log-return volatility was well-captured, the tendency for cumulative paths to sometimes exhibit wider dispersion or persistent drifts indicates a need for mechanisms that more strongly govern these longer-term characteristics. A promising avenue is to implement an explicit drift or mean regularization penalty within the generator's loss function. For instance, an L2 penalty could be applied to the difference between the mean endpoint of a batch of generated paths and a target mean derived from the real data distribution, thereby directly encouraging the generator to produce paths that, on average, end up in a more realistic location. However, such regularization must be carefully calibrated. Over-penalizing the endpoint mean could cause the generator to under-express variance leading to overly homogeneous trajectories. This reflects a broader trade-off observed in adversarial training between distributional realism and statistical constraints. Future work should therefore explore adaptive regularization strategies to ensure that global path realism is achieved.

Simultaneously, the discriminator's ability to perceive and penalize such global path deviations could be enhanced. This might involve incorporating features derived from path signatures, the powerful mathematical descriptors of path structure, as auxiliary inputs to the discriminator's decision layers. These signatures provide a hierarchical, non-linear summary of a trajectory's geometric properties and temporal ordering and could offer discriminative information complementary to that extracted by its CDE dynamics. While this work already incorporates simple global descriptors such as the mean and standard deviation of the latent path to guide the discriminator, more expressive representations could further enhance its ability to detect path-level inconsistencies.

The core SDE parameterization and the neural networks defining its dynamics also offer rich avenues for exploration. To better capture potential non-stationarity in market behavior, the SDE's drift and diffusion scaling parameters (*scale_f*, *scale_g*) could be evolved from fixed learnable scalars into time-varying or even state-dependent functions. This would empower the model to learn, for instance, varying volatility intensities across different time periods or market regimes. Stability could be further strengthened by developing more sophisticated adaptive regularization or learnable clamping strategies for these sensitive SDE scales, preventing divergence while preserving expressiveness.

Moreover, the MLPs currently parameterizing the SDE's drift and diffusion could be augmented or replaced with more powerful and flexible sequence-processing architectures like Transformers or attention-based networks. Their ability to capture long-range dependencies might lead to more refined temporal dynamics and a better match in subtle autocorrelations and higher-order moments, while also requiring more careful tuning and refined architectural choices.

To push the boundaries of financial realism, future research could integrate more profound domain knowledge and advanced stochastic processes. Exploring SDEs that incorporate jump components, such as models driven by Lévy processes, would allow the generator to produce paths with sudden, discontinuous shocks characteristic of real market events, significantly improving the modeling of tail risk and extreme outcomes.

Another powerful direction is hybrid modeling, which seeks to integrate established econometric models (such as GARCH for volatility dynamics) directly into the SDE structure. This could provide strong inductive biases, potentially leading to more robust and financially interpretable generated series by synergizing data-driven deep learning with decades of financial theory. For instance, the conditional variance estimated by a GARCH model could guide the volatility structure in the SDE’s diffusion term, allowing the generator to better reflect empirically observed volatility dynamics.

Finally, to truly bridge the gap between statistical replication and practical financial utility, future work should focus on expanding the SDE-GAN's direct applicability and realism within relevant financial decision-making frameworks.

A crucial advancement would be the development of a conditional SDE-GAN. This involves extending the model to generate financial time series paths that are explicitly conditioned on specified external factors or market states. For instance, trajectories could be generated based on

projected macroeconomic scenarios (e.g., varying central bank interest rate policies, inflation forecasts based on CPI, or GDP growth outlooks), or current market sentiment indicators (such as prevailing levels of implied volatility indices, or quantitative scores derived from financial news sentiment). A robust conditional model would significantly elevate the SDE-GAN's relevance for sophisticated scenario analysis, regulatory stress testing, the design of dynamic asset allocation or hedging strategies that must adapt to changing economic environments.

Another critical extension is progressing towards multivariate financial time series generation. While the current model adeptly handles univariate series, real-world financial markets are characterized by the complex interplay of numerous assets. A multivariate SDE-GAN would aim to capture not only the individual stylized facts of multiple assets (e.g., within a diverse portfolio of equities, bonds, and commodities) but also their crucial cross-asset correlation structures and, ideally, more intricate co-dependencies such as volatility spillovers or potential cointegration relationships. This capability is fundamental for practical applications in modern portfolio construction, robust risk diversification analysis, statistical arbitrage, and the development of valuable trading strategies.

The ultimate test, however, for both these future directions and the current model's accomplishments, is assessing how useful the generated data proves to be in actual financial applications. This involves moving beyond statistical comparisons to evaluate performance in practical use-cases. The generated trajectories can serve as inputs for real world risk assessment tasks. For example, they could be employed in Monte Carlo simulations for robust estimation of tail risk measures like Value-at-Risk (VaR) and Expected Shortfall (ES), which rely on empirical distribution of returns; or for stress-testing the out-of-sample performance of trading strategies across a broader range of scenarios than historical data alone allows.
Demonstrating efficacy in such concrete applications would offer more tangible proof of the SDE-GAN's value as a sophisticated tool for financial modeling, risk management, and decision support.

Bibliography

Applebaum, D. (2009). *Lévy Processes and Stochastic Calculus* (2nd ed.). Cambridge University Press. DOI: 10.1017/CBO9780511809781

Arjovsky, M., Chintala, S., & Bottou, L. (2017). Wasserstein GAN. *arXiv preprint arXiv:1701.07875*.

Bartosh, G., Vetrov, D., & Naesseth, C. A. (2025). *SDE Matching: Scalable and Simulation-Free Training of Latent Stochastic Differential Equations*. arXiv preprint arXiv:2502.02472v1 [cs.LG].

Bento, C. (2021). Multilayer Perceptron explained with a real-life example and Python code: Sentiment analysis. Medium. https://medium.com/data-science/multilayer-perceptron-explained-with-a-real-life-example-and-python-code-sentiment-analysis-cb408ee93141

Black, F., & Scholes, M. (1973). The Pricing of Options and Corporate Liabilities. *Journal of Political Economy, 81*(3), 637-654.

Chen, R. T. Q., Rubanova, Y., Bettencourt, J., & Duvenaud, D. K. (2018). Neural Ordinary Differential Equations. *Advances in Neural Information Processing Systems (NeurIPS)*, 31.

Cont, R. (2001). Empirical properties of asset returns: stylized facts and statistical issues. *Quantitative Finance, 1*(2), 223-236.

Cox, J. C., Ingersoll, J. E., & Ross, S. A. (1985). A Theory of the Term Structure of Interest Rates. *Econometrica, 53*(2), 385-407.

Esteban, C., Hyland, S. L., & Rätsch, G. (2017). Real-valued (medical) time series generation with recurrent conditional GANs. *arXiv preprint arXiv:1706.02633*.

Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., ... & Bengio, Y. (2014). Generative Adversarial Nets. *Advances in Neural Information Processing Systems 27 (NIPS 2014)*.

Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V., & Courville, A. C. (2017). Improved Training of Wasserstein GANs. *Advances in Neural Information Processing Systems (NeurIPS)*, 30. arXiv: 1704.00028

Heston, S. L. (1993). A Closed-Form Solution for Options with Stochastic Volatility with Applications to Bond and Currency Options. *Review of Financial Studies, 6*(2), 327-343.

Jia, J., & Benson, A. R. (2019). Neural Jump Stochastic Differential Equations. *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*.

Karatzas, I., & Shreve, S. E. (1991). *Brownian Motion and Stochastic Calculus*. Springer.

Kidger, P., Morrill, J., Foster, J., & Lyons, T. J. (2020). Neural Controlled Differential Equations for Irregular Time Series. *Advances in Neural Information Processing Systems 33 (NeurIPS 2020). Github: https://github.com/patrick-kidger/torchcde*

Kidger, P., Foster, J., Li, X., Oberhauser, H., & Lyons, T. (2021). Neural SDEs as Infinite-Dimensional GANs. *Proceedings of the 38th International Conference on Machine Learning (ICML)* Github:https://github.com/googleresearch/torchsde/blob/master/examples/sde_gan.py

Kloeden, P. E., & Platen, E. (1992). *Numerical Solution of Stochastic Differential Equations*. Springer.

Kwossek, A. P., Prömel, D. J., & Teichmann, J. (2025). *Universal approximation property of neural stochastic differential equations*. arXiv preprint arXiv:2503.16696 [math.PR].

Li, X., Wong, T. K. L., Chen, R. T. Q., & Duvenaud, D. (2020). Scalable Gradients for Stochastic Differential Equations. *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics (AISTATS 2020)*

Liao, S., Sabaté-Vidales, M., Wiese, M., Ni, H., & Salvi, C. (2023). *Sig-Wasserstein GANs for conditional time series generation*. Mathematical Finance, 33(1), 211–250.

Little, C., Elliot, M., Allmendinger, R., & Shariati Samani, S. (2021). Generative Adversarial Networks for Synthetic Data Generation: A Comparative Study (arXiv preprint arXiv:2112.01925)

Liu, T. Xiao, S. Si, Q. Cao, S. Kumar, and C.-J. Hsieh. (2019) Neural SDE: Stabilizing Neural ODE Networks with Stochastic Noise. arXiv:1906.02355.

Merton, R. C. (1973). Theory of Rational Option Pricing. *Bell Journal of Economics and Management Science, 4*(1), 141-183.

Mogren, O. (2016). C-RNN-GAN: Continuous recurrent neural networks with adversarial training. *arXiv preprint arXiv:1611.09904*.

Ni, H., Szpruch, L., Sabate-Vidales, M., Xiao, B., Wiese, M., & Liao, S. (2021). *Sig-Wasserstein GANs for Time Series Generation*. arXiv preprint arXiv:2111.01207v1

Oh, Y., Lim, D.-Y., & Kim, S. (2024). *Stable Neural Stochastic Differential Equations in Analyzing Irregular Time Series Data*. arXiv preprint arXiv:2402.14989v1 [cs.LG].

Øksendal, B. K. (2003). *Stochastic Differential Equations: An Introduction with Applications*. Springer.

Papanicolaou, A. (2019). Introduction to Stochastic Differential Equations (SDEs) for Finance. arXiv preprint arXiv:1504.05309.

Rackauckas, C., Ma, Y., Martensen, J., Warner, C., Zubov, K., Supekar, R., ... & Edelman, A. (2020). Universal Differential Equations for Scientific Machine Learning. *arXiv preprint arXiv:2001.04385*.

R. T. Q. Chen. torchdiffeq, 2018. https://github.com/rtqichen/torchdiffeq.

Shen, M., & Cheng, C. (2025). *Neural SDEs as a Unified Approach to Continuous-Domain Sequence Modeling*. arXiv preprint arXiv:2501.18871v1 [cs.LG].

Snow, L., & Krishnamurthy, V. (2025). *Efficient Neural SDE Training using Wiener-Space Cubature*. arXiv preprint arXiv:2502.12395v2 [cs.LG].

Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., & Poole, B. (2021). Score-Based Generative Modeling through Stochastic Differential Equations. *International Conference on Learning Representations (ICLR 2021)*.

Shuai Zhang, Chuan Zhou, Yang Aron Liu, Peng Zhang, Xixun Lin, and Zhi-Ming Ma. (2024) Neural jump-diffusion temporal point processes. In Forty-first International Conference on Machine Learning, 2024.

Tanabe, R., & Koga, K. N. (2024). *Stable Neural SDEs*. arXiv preprint arXiv:2402.14989. GitHub. Retrieved from https://github.com/yongkyung-oh/Stable-Neural-SDEs

Tzen, B., & Raginsky, M. (2019). Neural Stochastic Differential Equations: Deep Latent Gaussian Models. *arXiv preprint arXiv:1905.09883*.

Vaisband, M., von Bornhaupt, V., Schmid, N., Abulizi, I., & Hasenauer, J. (2023). Loss formulations for assumption-free neural inference of SDE coefficient functions. *Machine Learning*, *112*(11), 4461-4491.

Vasicek, O. (1977). An equilibrium characterisation of the term structure. *Journal of Financial Economics*, *5*(2), 177-188.

Yang, L., Gao, T., Lu, Y., Duan, J., & Liu, T. (2022). Neural network stochastic differential equation models with applications to financial data forecasting. *Chaos, Solitons & Fractals*, *160*, 112224.

Yoon, J., Jarrett, D., & van der Schaar, M. (2019). Time-series Generative Adversarial Networks. *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*.

Zhang, J., Viktorov, J., Jung, D., & Pitler, E. (2024). *Efficient Training of Neural Stochastic Differential Equations by Matching Finite Dimensional Distributions*. arXiv preprint arXiv:2402.06065.